

GST BILLING AND MANAGEMENT SYSTEM

Submitted by

PRITI PRIYA

For the award of the degree of

**BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE-ARTIFICIAL INTELLIGENCE**

Under the supervision of
KULDEEP SINGH
FOUNDER AND CEO
KABADI TECHNO



Faculty of Mathematics and Computing

Banasthali Vidyapith

Banasthali – 304022

Session: 2026

CERTIFICATE

Certified that **PRITI PRIYA** has carried out the project work titled "GST Billing and Management System" from **12/01/2026** to **31/05/2026** for the award of the **Bachelors Of Technology – Artificial Intelligence** from **KABADI TECHNO** under my supervision.

The thesis embodies result of original work and studies carried out by the student herself and the contents of the thesis do not form the basis for the award of any other degree to the candidate or to anybody else.

(Signature of the Supervisor)

(Name of the Supervisor)

Designation: Founder and CEO

Place: Agra, Uttar pradesh

Date:11-06-2026

ABSTRACT

The GST Billing and Management System is a web-based application developed using Django REST Framework that automates and digitizes the Goods and Services Tax (GST) billing process for small and medium-sized enterprises (SMEs) in India. The system addresses the challenges of manual billing errors, GST non-compliance, and fragmented buyer-seller communication by providing a unified platform for GST-compliant invoicing.

The platform supports two types of users: GST-registered sellers who can create products, generate invoices, and manage GST returns; and Non-GST customers who can receive bills, track purchases, and use a digital wallet. Key modules include user authentication with email verification, JWT-based session management, product catalog management with HSN code and barcode support, automatic computation of CGST and SGST, PDF and Excel invoice export, a referral and credit wallet system, return and exchange management with GST reversal and credit note generation, bill verification workflows, an integrated payment transaction module, and automated monthly GST filing summary computation.

The system is built on REST API architecture, making it platform-independent and compatible with any web or mobile frontend. The database handles complex billing scenarios including partial returns, exchange bills, credit notes, and bill-level fraud flagging. The project demonstrates practical application of software engineering principles including requirement analysis, system design, implementation, and testing in the domain of financial technology and e-governance.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor for their invaluable guidance, constant encouragement, and unwavering support throughout this project. Their technical expertise and insightful suggestions have been instrumental in shaping this work.

I am deeply thankful to the Faculty of Mathematics and Computing, Banasthali Vidyapith, for providing a conducive academic environment and access to the resources that made this project possible.

I extend my heartfelt thanks to my colleagues and batchmates for their stimulating discussions and feedback during the testing phases of this project.

I also acknowledge the open-source community, especially the developers of Django, Django REST Framework, ReportLab, and Python-Barcode, whose tools formed the technical foundation of this project.

Finally, I am forever grateful to my family for their patience, love, and encouragement, without which this endeavor would not have been possible.

Priti priya
Banasthali Vidyapith
Session: 2026

TABLE OF CONTENTS

Chapter 1 Objective (Problem Statement)	1
Chapter 2 Requirement Analysis (SRS)	7
2.1 Requirement Specification.....	7
2.2 Hardware and Software Requirements	12
2.3 Feasibility Study	14
2.4 Product Functions	16
2.5 Use-Case Diagrams.....	18
Chapter 3 System Design (SDS)	26
3.1 High-Level Design.....	26
3.2 ER Diagram / Class Diagrams	29
3.3 Database Design.....	33
3.4 Data Flow Diagrams / Activity Diagrams	42
3.5 Flowcharts / Sequence Diagrams.....	46
Chapter 4 Coding – Main Modules	52
Chapter 5 Testing	64
5.1 Test Cases	64
Chapter 6 User Interfaces	70
Chapter 7 Appendices	74
Chapter 8 References and Bibliography	80

CHAPTER 1: OBJECTIVE (PROBLEM STATEMENT)

1.1 Introduction

The Goods and Services Tax (GST), introduced in India on 1st July 2017, unified a complex system of indirect taxes into a single national tax regime. While GST simplified the taxation framework at a policy level, its practical implementation by small and medium enterprises (SMEs) has remained challenging. Manual billing procedures, the absence of real-time GST rate validation, difficulty in maintaining accurate CGST and SGST records, and the lack of a unified digital platform for tax-compliant invoicing remain persistent problems for businesses across India.

The GST Billing and Management System aims to bridge this gap by providing a robust, digitized platform where GST-registered sellers and Non-GST customers can engage in GST-compliant transactions. The system automates invoice generation, GST computation, payment recording, and return-and-exchange workflows, ensuring end-to-end compliance with the GST Act.

1.2 Problem Statement

Small businesses and retail establishments in India face several recurring challenges in GST billing:

- Manual computation of CGST and SGST on each bill item leads to arithmetic errors and potential non-compliance with the GST Act, resulting in penalties.
- There is no unified digital identity system linking buyers and sellers, making end-to-end traceability of transactions difficult.
- Return, exchange, and replacement workflows are handled informally, without proper GST reversal documentation such as credit notes, creating reconciliation gaps.
- Monthly GST filing requires manual aggregation of sale-level data, which is time-consuming and error-prone for businesses handling hundreds of invoices.

- There is no automated fraud detection mechanism to flag bills where abnormal GST rates have been applied on products, enabling potential tax evasion.
- Buyers lack a centralized view of their purchase history, making return claims and billing disputes difficult to manage and resolve.
- Payment reconciliation across multiple modes (cash, UPI, card, credit points) has no structured digital tracking system for small businesses.

1.3 Objectives of the Project

The primary objectives of this project are:

1. To develop a web-based REST API platform for GST billing that supports both GST-registered sellers and Non-GST customers within a single integrated system.
2. To automate CGST and SGST computation based on HSN codes and a GST master rate database, and to validate applied GST rates against master data on every bill item.
3. To implement a complete user management system with email-verified registration, JWT authentication, dual-account support (GST and Non-GST), and GST profile management including GSTIN format verification.
4. To provide a product catalog system with barcode and HSN code support, price suggestions from GST master data, brand information, and product image storage.
5. To generate GST-compliant invoices in both PDF and Excel formats, showing itemized taxable values, CGST, SGST, total GST, discount amounts, and grand totals.
6. To build a return, exchange, and replacement management system with proportional GST reversal computation and automatic credit note generation in CRN-YYYY-XXX format.
7. To implement a bill verification workflow enabling sellers to request buyer digital consent before a bill is finalized and marked as paid.

8. To create a digital wallet system with credit points, signup bonuses (100 credits), referral rewards (50 credits), and credit-based payment support for Non-GST customers.
9. To implement monthly GST filing data aggregation, enabling sellers to view period-wise CGST and SGST totals for tax submission preparation.
10. To integrate a payment transaction tracking module supporting CASH, UPI, CARD, and CREDIT payment modes with seller and buyer confirmation workflows.

1.4 Scope of the Project

The scope of this project covers the following functional areas:

- A backend REST API application built using Python 3.9, Django 4.2, and Django REST Framework, deployable on any WSGI-compatible server.
- User authentication and authorization using JWT (JSON Web Tokens) via the SimpleJWT library, with support for token refresh and blacklisting on logout.
- A billing module covering product catalog management, bill creation with itemized GST computation, bill lifecycle management (CREATED, PAID, RETURNED, EXCHANGED), and invoice export in PDF and Excel.
- A payment module for recording and managing transactions across multiple payment modes with buyer and seller confirmation tracking.
- A return management module supporting full and partial returns with automatic proportional GST reversal and credit note generation.
- A referral and credit wallet system with tracking of credit earnings, expenditures, and transaction history.
- A notification system for alerting users of billing events, verification requests, and return status updates.
- Monthly GST filing summary reports for registered sellers, including total sales, CGST collected, SGST collected, and bill count per period.

The project explicitly does not include a frontend user interface (it exposes REST APIs for integration with any client application), live payment gateway integration (payment transactions are recorded locally but not processed externally), or actual submission of data to the GSTN portal.

1.5 Motivation

The project is motivated by the vision of Digital India and the government's push to make GST compliance accessible to every business, regardless of scale. With over 1.4 crore GST-registered businesses in India as of 2024, even a minor improvement in billing efficiency and accuracy translates to significant economic benefit at a national scale. Errors in GST computation, delayed filing, and inadequate documentation lead to penalties that disproportionately affect small traders and shopkeepers who lack the resources to employ dedicated accountants.

This system aims to empower shopkeepers, retailers, and service providers with the tools they need to issue accurate, compliant bills and manage their tax obligations with minimal effort and maximum transparency. By automating CGST/SGST computation, validating rates against the official GST master, and generating credit notes for returns, the system eliminates the most common sources of GST-related errors for SMEs.

1.6 Technologies Used

Technology	Purpose
Python 3.9+	Core programming language
Django 4.2	Web framework and ORM
Django REST Framework	REST API development
SimpleJWT	JWT authentication
ReportLab	PDF invoice generation
openpyxl / pandas	Excel export
qrcode	QR code generation
python-barcode	Barcode generation
SQLite / PostgreSQL	Relational database

1.7 Background of GST in India

Before the introduction of GST, India's indirect tax structure was fragmented across central and state taxes including Central Excise Duty, Service Tax, VAT, Entry Tax, Octroi, and several other levies. This multiplicity of taxes led to a cascading tax effect — commonly known as 'tax on tax' — where taxes were levied at each stage of the supply chain without adequate credit for taxes paid at earlier stages. The result was increased costs for businesses, compliance burden, and reduced competitiveness of Indian goods in global markets.

The GST Council, chaired by the Union Finance Minister and comprising finance ministers of all states, was constituted under Article 279A of the Constitution to make recommendations on GST-related matters. The Council adopted a dual GST model where both the Centre and States simultaneously levy taxes on the same transaction. For intra-state supplies, both CGST (Central GST) and SGST (State GST) are levied. For inter-state supplies, IGST (Integrated GST) is levied by the Centre and shared with the destination state.

The GST regime introduced four primary tax slabs: 5%, 12%, 18%, and 28%, along with a special 0% rate for essential commodities. Each good or service is classified under a specific HSN (Harmonised System of Nomenclature) code for goods or SAC (Services Accounting Code) for services. These codes are internationally standardized and form the backbone of GST rate determination — which is exactly why this project uses HSN codes in its product and billing modules to ensure accurate GST rate lookup and validation.

1.8 Need for Digital Billing Solutions

The GST Network (GSTN) is the IT backbone of the GST system in India. While large enterprises have enterprise resource planning (ERP) systems that integrate with GSTN for automated filing, small and medium businesses typically rely on manual calculations, spreadsheets, or basic billing software that may not enforce GST compliance rules. This gap between large enterprises and SMEs in terms of GST technology adoption is one of the primary motivations for this project.

The E-invoicing mandate introduced by the Government of India for businesses with turnover above Rs. 5 crores makes it mandatory to generate invoices through the GST portal or compatible software. This trend toward mandatory digital invoicing further underscores the relevance of a system like the GST Billing and Management System, which can serve as a compliant digital billing platform for businesses that need to adapt to these regulatory requirements.

Furthermore, the growing adoption of UPI (Unified Payments Interface) for retail payments in India makes it increasingly important for billing systems to track multiple payment modes in an integrated manner. This project's support for CASH, UPI, CARD, and CREDIT payment modes directly addresses this need, making it relevant for the current Indian retail payment landscape.

1.9 Comparison with Existing Systems

Feature	Tally ERP 9	Vyapar App	This Project
Platform	Desktop (Windows)	Mobile / Desktop	REST API (any platform)
GST Computation	Yes	Yes	Yes (CGST/SGST auto-split)
GST Rate Validation	No	No	Yes (against master database)
Bill Verification Workflow	No	No	Yes (buyer digital consent)
Credit Note / GST Reversal	Yes	Partial	Yes (proportional reversal)
Digital Wallet / Credits	No	No	Yes (100 signup + referral)
Referral System	No	No	Yes
REST API	No	No	Yes (platform-independent)
Open Source	No	No	Yes (Django stack)
Barcode / QR Support	Limited	Yes	Yes (auto-generated)

CHAPTER 2: REQUIREMENT ANALYSIS (SRS)

2.1 Requirement Specification

2.1.1 Functional Requirements

User Management Module:

- The system shall allow users to register with email, full name, phone number, and account type (GST or Non-GST).
- The system shall send an email verification link upon registration and activate the account only after the user clicks the link. Verification tokens shall expire after 24 hours.
- The system shall support JWT-based login and logout with refresh token blacklisting on logout.
- The system shall allow users to reset their password via a time-limited (1 hour) email token.
- The system shall allow GST users to complete a business profile including GSTIN, business name, business type, address, state, city, and pincode.
- The system shall validate GSTIN format before allowing a seller to create products or bills.
- The system shall generate a unique Bill ID (e.g., priti@bill) for each user as a digital billing address.
- The system shall automatically generate QR codes and barcodes for Non-GST users based on their Bill ID upon account activation.
- The system shall support a referral program where the referrer earns 50 credit points when a referred user completes their first purchase.

Billing Module:

- The system shall allow verified GST sellers to add products with name, brand name, price, GST percentage, barcode, HSN code, and product image.
- The system shall compute CGST and SGST as equal halves of the total GST percentage for each bill item for intra-state transactions.

- GST computation formula: Taxable Value = Quantity x Price. GST Amount = Taxable Value x (GST%/100). CGST = GST Amount/2. SGST = GST Amount/2. Total Price = Taxable Value + GST Amount.
- The system shall generate unique invoice numbers in the sequential format INV-XXXX for each bill.
- The system shall allow sellers to apply a percentage discount at the bill level, reducing the final total.
- The system shall validate each item's applied GST rate against the master database: Difference = 0 means VALID; Difference 1-5% means WARNING; Difference >5% means INVALID and the bill is flagged.
- The system shall support payment modes: CASH, UPI, CARD, and CREDIT.
- The system shall generate bill PDFs with seller GSTIN, seller name, buyer details, invoice number, date, itemized table with HSN codes, taxable value, CGST, SGST, and grand total.
- The system shall export bill data to Excel format with all item-level and summary data.

Return, Exchange, and Replacement Module:

- The system shall allow customers to raise return, exchange, or replacement requests specifying the original bill, selected items, quantities, and reason.
- GST Reversal formula: proportion = returned_qty / original_qty. GST Reversal = proportion x original GST amount. CGST and SGST reversal shall each equal half the total GST reversal.
- The system shall auto-generate credit notes in the format CRN-YYYY-XXX upon return completion by the seller.
- The system shall support partial returns where only some items or partial quantities are returned.

Wallet and Referral Module:

- The system shall credit 100 points to new Non-GST users as a signup bonus automatically upon registration.
- The system shall allow credit points to be used as a payment mode for bills, deducting from the customer's wallet.
- The system shall maintain a full transaction history for all wallet credits and debits.

GST Filing Module:

- The system shall aggregate total sales, CGST collected, and SGST collected by month/year for each GST seller.
- The system shall present the GST filing summary per period for seller review and tax preparation.

2.1.2 Non-Functional Requirements

- Security: All endpoints except registration and login shall require JWT Bearer token authentication. Passwords shall be stored using Django's PBKDF2 cryptographic hash algorithm. Critical bill fields shall be locked after payment.
- Performance: The system shall respond to all standard API requests within 2 seconds under normal load conditions.
- Reliability: Once a bill reaches PAID status, critical financial fields (total amount, GST components, seller, customer) shall be protected against unauthorized modification.
- Scalability: The REST API architecture shall allow horizontal scaling by adding application server nodes without requiring code changes.
- Portability: The system shall be deployable on any platform supporting Python 3.9+ with either SQLite (development) or PostgreSQL (production).
- Maintainability: The codebase shall be organized into separate Django apps (accounts, billing, payments) each with independent models, views, serializers, and URL configurations following Django's MTV pattern.

- Usability: All API responses shall follow consistent JSON structure with success/error flags and human-readable messages.

2.2 Hardware and Software Requirements

Development Hardware Requirements:

Component	Specification
Processor	Intel Core i5 or equivalent (2 GHz or above)
RAM	Minimum 4 GB (8 GB recommended)
Storage	Minimum 20 GB free disk space
Network	Internet connectivity for email services and testing
Display	1366x768 resolution or higher

Production Server Requirements:

Component	Specification
Processor	Dual-core server-grade CPU or higher
RAM	Minimum 2 GB (4 GB recommended for concurrent users)
Storage	20 GB SSD minimum for database and media files
Operating System	Ubuntu 20.04 LTS or CentOS 8
Web Server	Nginx (reverse proxy) + Gunicorn (WSGI)

Software Requirements:

Software	Version / Details
Python	3.9 or higher
Django	4.2.x
Django REST Framework	3.x
djangorestframework-simplejwt	JWT authentication tokens
ReportLab	PDF invoice generation
openpyxl	Excel file generation
pandas	Data processing for Excel exports
qrcode	QR code image generation
python-barcode	Barcode image generation (Code128)
Pillow	Image processing (required by qrcode)
dj-database-url	Database URL parsing for deployment
django-cors-headers	CORS headers for frontend integration
SQLite	Development and testing database
PostgreSQL	Production database
Git	Version control
Postman	API testing and documentation
VS Code / PyCharm	Development IDE

2.3 Feasibility Study

2.3.1 Technical Feasibility

The project uses Python and Django, which are well-established and extensively documented technologies with large global community support. Django REST Framework is a mature library used in production by thousands of organizations worldwide. SimpleJWT handles token authentication with comprehensive security features. All third-party libraries used (ReportLab, qrcode, python-barcode,

openpyxl, pandas) are mature, actively maintained open-source packages with stable APIs. The developer has access to all required tools and the technical knowledge to implement and deploy the system. The project is technically feasible.

2.3.2 Economic Feasibility

The entire technology stack is open-source and incurs no licensing costs. Development requires only a standard developer workstation and internet connectivity. Deployment can be accomplished on cloud platforms such as AWS EC2, DigitalOcean Droplets, or Heroku at a minimal monthly cost starting at approximately Rs. 500-1500 per month for a small-scale deployment. For businesses, the system reduces billing errors, prevents GST-related penalties, and eliminates the cost of manual data entry for monthly filing. The project is economically feasible and provides strong return on investment.

2.3.3 Operational Feasibility

The REST API architecture ensures the system can be integrated with any frontend — web application, mobile application, or desktop software — making it operationally flexible for any technology stack. GST sellers can perform all billing operations through a straightforward workflow: add products to catalog, scan customer barcodes, create bills, accept payment. Non-GST customers simply share their Bill ID or QR code to receive invoices and track their purchase history. The learning curve is minimal and the system is operationally feasible for deployment in small and medium businesses.

2.3.4 Schedule Feasibility

The project was developed over a defined project period following an iterative development approach. The modular architecture of Django applications (accounts, billing, payments) enabled parallel and independent development of different functional modules. Unit testing of each module was performed incrementally before integration. Given the defined scope and the available development timeline, the project schedule was feasible and the development was completed within the planned timeframe.

2.4 Product Functions

The following table summarizes all major functional modules and their actors:

Module	Sub-Module	Actor	Description
User Management	Registration & Login	All Users	JWT-based registration with email verification, login, and logout
User Management	GST Profile	GST Seller	GSTIN verification, business profile completion
User Management	QR Code & Barcode	Non-GST Customer	Auto-generated QR code and barcode tied to unique Bill ID
User Management	Notifications	All Users	Event-based notifications for billing, returns, and verifications
Product Management	Add / Update / Delete	GST Seller	Products with HSN code, barcode, GST percentage, and image
Product Management	GST Master Lookup	GST Seller	Look up HSN code to retrieve standard GST rate and price suggestion
Billing	Create Invoice	GST Seller	GST-compliant invoice with itemized CGST and SGST computation
Billing	Bill Verification	Both	Seller requests buyer consent; buyer approves or rejects via notification
Billing	PDF Export	Both	Download GST-compliant invoice as a formatted PDF file
Billing	Excel Export	Both	Download bill data as Excel spreadsheet with all item details
Payment	Record Payment	GST Seller	Record CASH, UPI, CARD, or CREDIT payment transactions
Return Management	Raise Return	Customer	Request return, exchange, or replacement with item-level selection
Return Management	Approve/Reject	GST Seller	Process return and compute proportional GST reversal per item
Return Management	Credit Note	System	Auto-generate credit note (CRN-YYYY-XXX) on return completion
Wallet	Credit Points	Customer	Earn signup bonus, referral credits; spend on bills; view history
Referral	Referral Tracking	All Users	Unique referral codes; 50 bonus credits on referred user first purchase
GST Filing	Monthly Summary	GST Seller	Aggregate CGST, SGST, bill count by month/year for filing
Dashboard	Seller Analytics	GST Seller	Total sales, pending bills, recent transactions, GST liability overview

2.5 Data Dictionary (Reference)

The data dictionary defines the meaning, type, and constraints for all key data elements used in the system.

Data Element	Type	Valid Values / Format	Description
bill_id	String	<name>@bill, max 100 chars	Unique billing identity of a user e.g. priti@bill
invoice_number	String	INV-0001 to INV-9999	Sequential invoice identifier assigned per bill
credit_note_number	String	CRN-YYYY-XXX	Credit note identifier e.g. CRN-2026-001
gstn	String	15-char alphanumeric	GST Identification Number in standard format
hsn_code	String	4 to 8 digits	Harmonised System of Nomenclature product code
gst_percentage	Float	0.0, 5.0, 12.0, 18.0, 28.0	GST rate applicable on a product
taxable_value	Float	≥ 0.00	Product amount before GST = Qty x Price
cgst	Float	≥ 0.00	Central GST = $\text{gst_amount} / 2$
sgst	Float	≥ 0.00	State GST = $\text{gst_amount} / 2$
payment_status	Enum	CREATED, PAID, RETURN_REQUESTED, RETURNED, PARTIALLY_RETURNED, EXCHANGED, REVERSED	Current lifecycle state of a bill
gst_status	Enum	PENDING, VALID, WARNING, INVALID	GST rate validation result for a bill
account_type	Enum	GST, Non-GST	Type of user account
payment_mode	Enum	CASH, UPI, CARD, CREDIT	Payment method used for a bill
request_type	Enum	RETURN, EXCHANGE, REPLACEMENT	Type of return request raised
wallet_balance	Integer	≥ 0	Credit points balance in user wallet
month_year	String	YYYY-MM format	Filing period e.g. 2026-04

2.6 System Constraints

The following constraints apply to the GST Billing and Management System:

- A bill cannot be created by a GST seller for themselves — the seller and customer must be different users.
- A product can only be managed (created, updated, deleted) by the seller who owns it.
- GST computation always assumes intra-state transactions (CGST + SGST split). IGST for inter-state transactions is outside the current scope.
- Once a bill reaches PAID status, its critical financial fields (total_amount, total_cgst, total_sgst, seller, customer) are locked against modification.
- Return requests can only be raised on bills with payment_status = PAID. Unpaid or already-returned bills cannot be returned again.
- CREDIT payment mode is only available when the customer's wallet balance is sufficient to cover the full bill amount. Partial credit payment is not supported.
- The system stores monetary values as FLOAT with 2 decimal place rounding applied in application logic. All financial computations use Python's round() function with ndigits=2.
- A Non-GST customer cannot create bills or add products. Billing is exclusively performed by GST sellers.
- Email verification tokens expire after 24 hours. Password reset tokens expire after 1 hour. Expired tokens cannot be reused.

2.7 Assumptions

The following assumptions have been made during the design and development of this system:

1. All transactions are intra-state. CGST and SGST are always applied as equal halves of the total GST rate. IGST is not implemented in this version.
2. Product prices entered by sellers are exclusive of GST (tax-exclusive pricing model). GST is computed on top of the entered price.

3. The GST master rate database is pre-loaded with HSN codes and standard GST rates. Real-time synchronization with the GSTN portal is not implemented.
4. All monetary values are in Indian Rupees (INR). Multi-currency support is outside the scope of this project.
5. The referral bonus of 50 credits is awarded to the referrer when the referred user makes their first purchase. This is tracked by monitoring the customer's first PAID bill.
6. Email service (SMTP) credentials are configured in Django settings. The system assumes a working SMTP server is available for activation and notification emails.
7. File storage for product images, QR codes, and barcodes uses Django's default MEDIA_ROOT configuration. Cloud storage (S3, GCS) integration is not implemented but can be added via django-storages.
8. The system is designed for single-currency, single-country (India) operation in alignment with Indian GST regulations.

2.8 Use Case Elaboration — Bill Verification Workflow

The bill verification workflow is a unique feature of this system designed to ensure buyer consent before a bill is finalized for UPI or CREDIT payment modes. The following elaborates this workflow in detail:

Trigger Condition: The seller creates a bill and selects UPI or CREDIT as the payment mode.

Step-by-Step Flow:

9. System creates a BillVerification record linked to the bill, seller, and customer with status PENDING.
10. System creates a Notification for the customer with the bill details and a verification token.
11. Customer receives the notification and reviews the bill details (items, quantities, prices, GST breakdown, total).
12. Customer can either APPROVE or REJECT the verification:

- If APPROVED: BillVerification.status = VERIFIED. The bill can now proceed to payment.
- If REJECTED: BillVerification.status = REJECTED. The seller is notified and can modify or cancel the bill.

13. For CASH and CARD payments, the verification step is skipped since the transaction is immediate and physical.

This workflow protects customers from unauthorized billing and provides a digital audit trail of buyer consent, which is particularly important for UPI transactions where payment can be initiated remotely

2.5 Use-Case Diagrams

2.5.1 Actors in the System

The system involves three primary actors who interact with different functional modules. The GST Seller is a GSTIN-registered business user who manages products, creates bills, approves returns, and exports reports. The Non-GST Customer is a buyer who receives bills, raises return requests, and uses wallet credits. The System Administrator manages the GST master rate database, HSN codes, and system configurations.

2.5.2 Overall System Use-Case Diagram

The diagram below shows all use cases in the system and how each actor interacts with them. Seller use cases are shown in purple, customer use cases in coral, admin use cases in amber, and system-triggered use cases in gray.



Figure 2.5.1: Overall Use-Case Diagram — GST Billing and Management System

2.5.3 Use-Case Descriptions

2.5.3.1 Use-Case: User Registration and Email Verification

Primary Actor: All Users. Pre-condition: User is not registered in the system.

Main Flow:

11. User submits registration form with name, email, phone, account type (GST or Non-GST), and password.
12. System validates all fields and checks whether an account with the same email and account type already exists.
13. System creates the user record and auto-generates a unique Bill ID and Referral Code.
14. If the account type is Non-GST, system creates a Wallet with 100 credit points and records a Signup Bonus transaction.

15. System creates an EmailVerificationToken valid for 24 hours and sends an activation email.
16. User clicks the activation link; system validates token expiry and marks the account as verified and active.
17. If the user registered with a valid referral code and this is their first purchase, the referrer receives 50 credit points.

Alternate Flow: If the activation token has expired, the user can request a new activation link from the re-send endpoint.

2.5.3.2 Use-Case: Create GST Invoice

Primary Actor: GST Seller. Pre-condition: Seller is logged in with a verified and completed GST profile.

Main Flow:

18. Seller provides the customer's Bill ID to identify the buyer.
19. Seller adds items to the bill by providing product ID, quantity, price, and GST percentage.
20. For each item, system computes: $\text{Taxable Value} = \text{Qty} \times \text{Price}$. $\text{GST Amount} = \text{Taxable} \times (\text{GST}\%/100)$. $\text{CGST} = \text{GST}/2$. $\text{SGST} = \text{GST}/2$.
21. System validates each item's GST rate against the master database and sets bill-level `gst_status` to VALID, WARNING, or INVALID.
22. Seller optionally applies a percentage discount. System computes: $\text{Discount Amount} = \text{Total} \times \text{Discount}\%$.
23. System saves the bill with a sequential invoice number (INV-XXXX) and status CREATED.
24. For UPI or CREDIT payments, system creates a verification notification for the customer.
25. Seller receives the invoice number, GST breakdown, and `gst_status` in the response.

Exception Flow: Seller cannot create a bill for themselves. Product ID must belong to the authenticated seller.

2.5.3.3 Use-Case: Return Request and Credit Note Generation

Primary Actor: Non-GST Customer. Pre-condition: Original bill has payment status = PAID.

Main Flow:

26. Customer raises a return request specifying the original bill ID, request type (RETURN/EXCHANGE/REPLACEMENT), items with quantities, and reason.
27. System marks the bill as RETURN_REQUESTED and creates a ReturnRequest record with status PENDING.
28. Seller reviews the request. If rejected: status becomes REJECTED, customer is notified, and bill status reverts.
29. On approval, seller marks the return as complete. System computes proportional GST reversal for each item.
30. System generates a Credit Note with number CRN-YYYY-XXX capturing taxable reversal, CGST reversal, and SGST reversal.
31. Bill status is updated to RETURNED (full) or PARTIALLY_RETURNED (partial).
32. For exchanges: a new exchange bill is created with exchange_parent referencing the original bill.
33. Customer is notified of return completion and any applicable refund or credits.

2.5.4 Use-Case: Complete System Interaction Summary

Use Case ID	Use Case Name	Actor	Description
UC-01	Register Account	All Users	Register with email, phone, account type; receive activation email
UC-02	Email Verification	System	Activate account via 24-hour token; grant Non-GST 100 credits
UC-03	Login	All Users	Authenticate with credentials; receive JWT access and refresh tokens
UC-04	Logout	All Users	Blacklist refresh token to invalidate the current session
UC-05	Reset Password	All Users	Request email link; reset password with 1-hour valid token
UC-06	Complete GST Profile	GST Seller	Enter GSTIN, business name, address, state to enable billing
UC-07	Add Product	GST Seller	Register product with HSN code, barcode, price, GST percentage
UC-08	Scan Barcode	GST Seller	Retrieve product details instantly by barcode lookup
UC-09	Create Invoice	GST Seller	Create GST invoice for customer with CGST/SGST computation

UC-10	Verification Request	GST Seller	Request customer digital approval before finalizing bill
UC-11	Verify Bill	Non-GST Customer	Approve or reject a pending bill verification notification
UC-12	Make Payment	Customer	Pay bill using CASH, UPI, CARD, or CREDIT mode
UC-13	Export PDF	Both	Download GST-compliant invoice as a formatted PDF document
UC-14	Export Excel	Both	Download bill data as Excel spreadsheet
UC-15	Raise Return	Customer	Request return, exchange, or replacement with item selection
UC-16	Approve Return	GST Seller	Approve return and trigger GST reversal computation
UC-17	Complete Return	GST Seller	Finalize return; system generates credit note automatically
UC-18	View Dashboard	GST Seller	View total sales, GST collected, pending bills, recent activity
UC-19	Purchase History	Customer	View all received bills and their current lifecycle status
UC-20	GST Filing Summary	GST Seller	View monthly CGST/SGST totals for tax submission preparation
UC-21	Manage Wallet	Customer	View balance, earn via referrals, spend credits on bills
UC-22	Manage GST Master	Admin	Upload and update HSN code to GST rate mappings

CHAPTER 3: SYSTEM DESIGN (SDS)

3.1 High-Level Design

The GST Billing and Management System is built on a three-tier architecture separating presentation, application logic, and data storage. This separation ensures the system is scalable, maintainable, and platform-independent.

Tier	Technology and Responsibility
Presentation Tier	REST API endpoints exposed via Django REST Framework. Clients communicate via HTTPS requests in JSON format. JWT Bearer tokens are passed in the Authorization header for all protected routes.

Application Tier	Django 4.2 application server with three apps: accounts (user management), billing (invoice and return logic), and payments (transaction management). JWT middleware authenticates all protected routes automatically.
Data Tier	SQLite (development) or PostgreSQL (production) relational database managed via Django ORM. Media files including product images, QR codes, and barcodes are stored in the file system under media/

System Architecture — High-Level Design

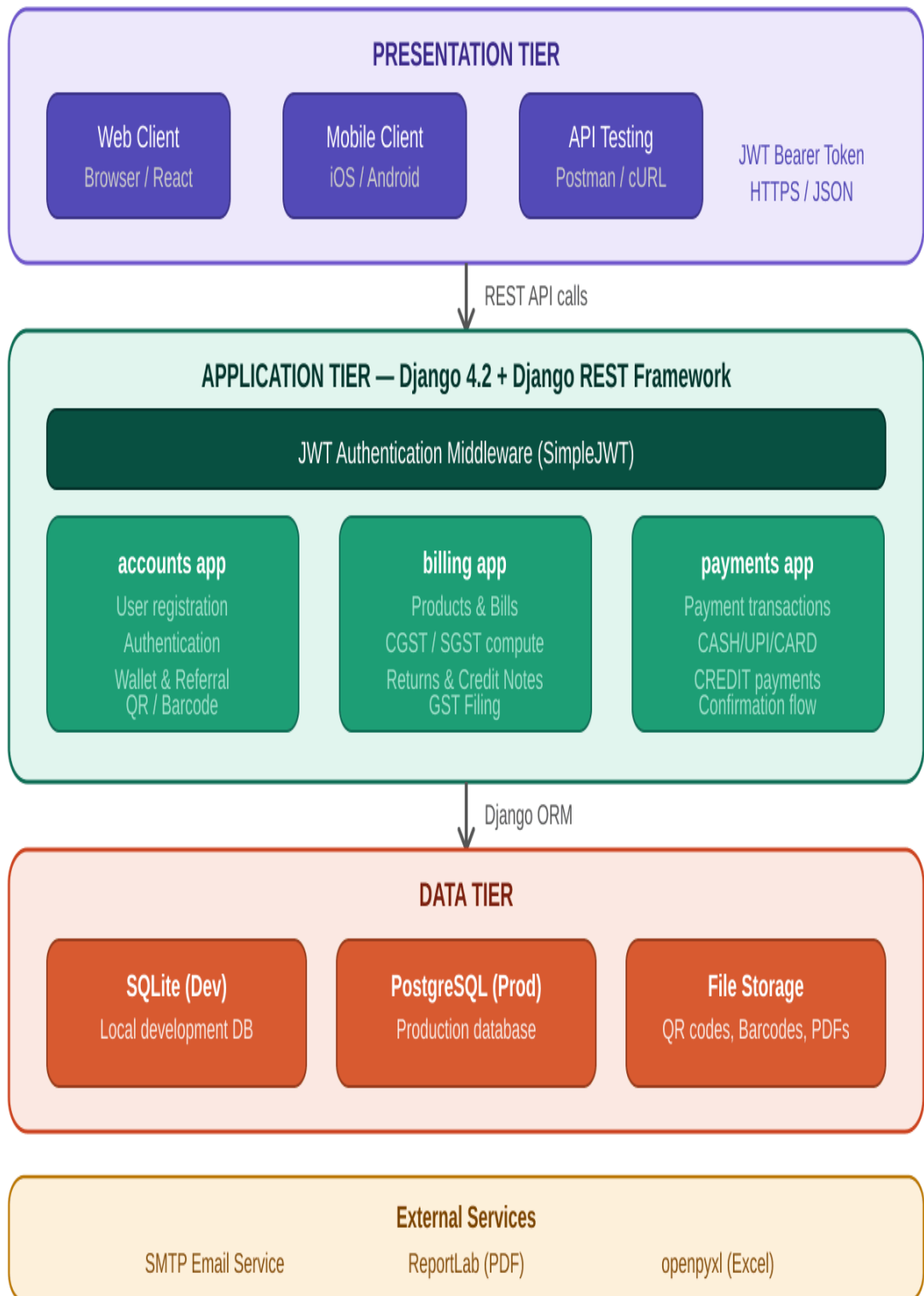


Figure 3.1: System Architecture — Three-Tier High-Level Design

Django Application Modules:

App	Responsibility	Key Models
accounts	User registration, JWT auth, profile management, wallet, referral, notifications, QR code and barcode generation	CustomUser, Wallet, CreditTransaction, Notification, EmailVerificationToken, PasswordResetToken
billing	Products, invoices, bill items, GST computation, returns, credit notes, GST master data, GST filing summaries	Product, Bill, BillItem, GSTMaster, GSTItem, BillVerification, ReturnRequest, ReturnItem, CreditNote, GSTFiling
payments	Payment transaction recording, multi-mode payment support, buyer/seller confirmation workflow	PaymentTransaction

API Request-Response Flow:

34. Client sends HTTPS request with JWT Bearer token in the Authorization header.
35. Django JWT middleware validates the token and attaches the authenticated user to the request object.
36. The API view executes business logic using Django ORM model operations and serializer validation.
37. Serializers validate incoming JSON data and serialize outgoing response data.
38. Django ORM executes parameterized SQL queries against the database.
39. Response is returned as JSON with appropriate HTTP status codes (200, 201, 400, 401, 403, 404).

3.2 ER Diagram / Class Diagrams

4.2.1 Entity Relationship Diagram

The ER diagram below shows all database entities, their key attributes, and the relationships between them. Primary and foreign keys are explicitly labeled. The diagram follows standard ER notation with entity boxes, attribute lists, and relationship cardinalities.

Entity Relationship Diagram (ER Diagram)

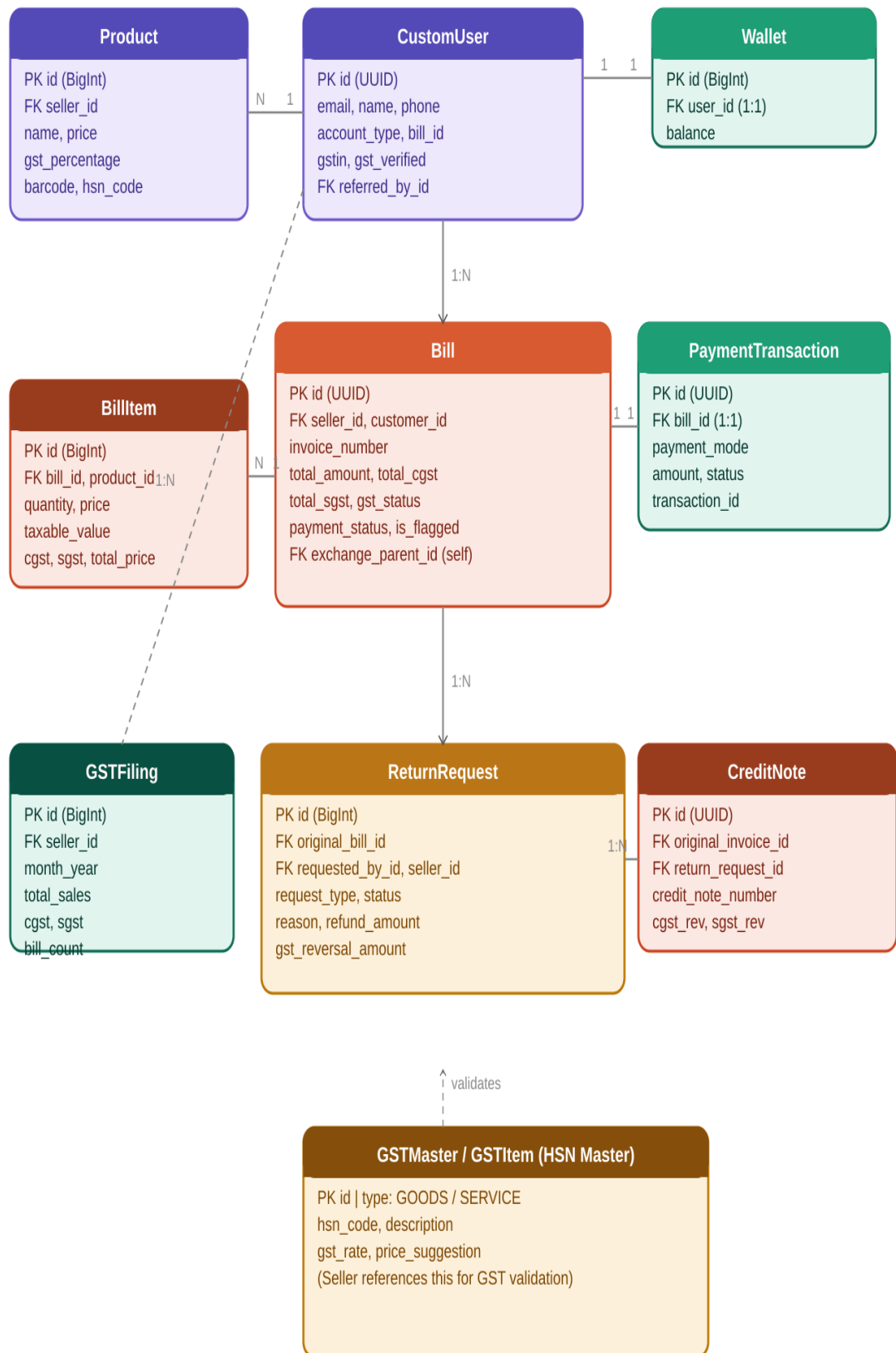


Figure 3.2: Entity Relationship Diagram — Database Entities and Relationships

4.2.2 Key Entity Relationships

- CustomUser has a one-to-one relationship with Wallet, and one-to-many relationships with Notification, CreditTransaction, Product, and Bills (as both seller and customer). It is self-referential for the referred_by relationship.
- Bill has a one-to-many relationship with BillItem and a one-to-one relationship with PaymentTransaction. It carries a self-referential foreign key exchange_parent that links exchange transactions back to their original bill.
- ReturnRequest references a Bill (original_bill) in a many-to-one relationship and aggregates ReturnItem records in a one-to-many relationship. It also has a one-to-many relationship with CreditNote.
- GSTMaster aggregates GSTItems in a one-to-many relationship, indexed by HSN code and type (GOODS or SERVICE). Bill validation checks item GST rates against this master.
- Product belongs to exactly one seller (CustomUser) in a many-to-one relationship, and is referenced by BillItem in a many-to-one relationship with SET NULL on delete.

3.3 Database Design

The following tables define the complete database schema. All UUID fields use Django's built-in UUIDField with auto-generation. All FLOAT fields store monetary values rounded to 2 decimal places in application logic.

3.3.1 Table: accounts_customuser

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, auto	Unique user identifier
email	VARCHAR	NOT NULL	Email address
name	VARCHAR(200)	NOT NULL	Full name
phone	VARCHAR(15)	NOT NULL	Mobile number
account_type	VARCHAR(20)	NOT NULL	GST or Non-GST
gsttin	VARCHAR(20)	NULL	GST Identification Number
business_name	VARCHAR(255)	NULL	Registered business name
bill_id	VARCHAR(100)	UNIQUE, NOT NULL	Unique billing address e.g. priti@bill
my_referral_code	VARCHAR(20)	UNIQUE, NULL	User's referral code
referred_by_id	UUID	FK CustomUser, NULL	Referrer user
gst_verified	BOOLEAN	DEFAULT FALSE	GST profile verified flag
profile_completed	BOOLEAN	DEFAULT FALSE	Business profile complete flag
is_email_verified	BOOLEAN	DEFAULT FALSE	Email verification status
qr_code	VARCHAR	NULL	QR code image file path
barcode	VARCHAR	NULL	Barcode image file path
created_at	DATETIME	auto_now_add	Registration timestamp

Unique constraint: (email, account_type) — same email may register as both GST and Non-GST separately.

3.3.2 Table: billing_product

Column	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, auto	Product ID
seller_id	UUID	FK CustomUser, CASCADE	Owning seller user
name	VARCHAR(255)	NOT NULL	Product display name
brand_name	VARCHAR(255)	NULL	Brand or manufacturer name
price	FLOAT	NOT NULL	Base price excluding GST
gst_percentage	FLOAT	NOT NULL	Applicable GST rate e.g. 18.0
barcode	VARCHAR(100)	NOT NULL	Product barcode value
hsn_code	VARCHAR(50)	NOT NULL	HSN or SAC code
image	VARCHAR	NULL	Product image file path
created_at	DATETIME	auto_now_add	Creation timestamp

3.3.3 Table: billing_bill

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Invoice UUID
seller_id	UUID	FK CustomUser	Bill creator (GST seller)
customer_id	UUID	FK CustomUser	Bill recipient (buyer)
invoice_number	VARCHAR(20)	UNIQUE	Sequential: INV-XXXX format
total_amount	FLOAT	DEFAULT 0	Grand total including GST minus discount
total_gst	FLOAT	DEFAULT 0	Total GST amount across all items
total_taxable_value	FLOAT	DEFAULT 0	Sum of taxable values excluding GST
total_cgst	FLOAT	DEFAULT 0	Central GST total
total_sgst	FLOAT	DEFAULT 0	State GST total
gst_status	VARCHAR(10)	DEFAULT PENDING	VALID / WARNING / INVALID
payment_mode	VARCHAR(10)	NULL	CASH / UPI / CARD / CREDIT
payment_status	VARCHAR(20)	DEFAULT CREATED	Bill lifecycle status
is_flagged	BOOLEAN	DEFAULT FALSE	Fraud detection flag for INVALID GST
discount_percentage	FLOAT	DEFAULT 0	Discount percentage applied
discount_amount	FLOAT	DEFAULT 0	Computed discount value in Rs.
exchange_parent_id	UUID	FK Bill, NULL	Original bill for exchange transactions
created_at	DATETIME	auto_now_add	Invoice creation timestamp

3.3.4 Table: billing_billitem

Column	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, auto	BillItem ID
bill_id	UUID	FK Bill, CASCADE	Parent bill
product_id	BIGINT	FK Product, SET NULL	Product reference
name	VARCHAR(255)	NOT NULL	Item name snapshot at billing time
quantity	INT	NOT NULL	Quantity purchased
price	FLOAT	NULL	Per-unit price at billing time
gst_percentage	FLOAT	NULL	GST rate applied on this item
taxable_value	FLOAT	DEFAULT 0	Quantity x Price
gst_amount	FLOAT	NOT NULL	Total GST amount for this line item
cgst	FLOAT	DEFAULT 0	CGST component = $\text{gst_amount} / 2$
sgst	FLOAT	DEFAULT 0	SGST component = $\text{gst_amount} / 2$
total_price	FLOAT	NOT NULL	Taxable Value + GST Amount

3.3.5 Table: billing_returnrequest

Column	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, auto	Return request ID
original_bill_id	UUID	FK Bill	Bill being returned
requested_by_id	UUID	FK CustomUser	Customer raising the request
seller_id	UUID	FK CustomUser	Seller assigned to process return
request_type	VARCHAR(20)	NOT NULL	RETURN / EXCHANGE / REPLACEMENT
status	VARCHAR(20)	DEFAULT PENDING	PENDING / APPROVED / REJECTED / COMPLETED
reason	TEXT	NOT NULL	Customer's reason for return
refund_amount	FLOAT	DEFAULT 0	Refund amount to be issued
gst_reversal_amount	FLOAT	DEFAULT 0	Total GST amount to be reversed
exchange_bill_id	UUID	FK Bill, NULL	New exchange bill reference
is_partial_return	BOOLEAN	DEFAULT FALSE	True if only some items returned
created_at	DATETIME	auto_now_add	Request creation timestamp

3.3.6 Table: billing_creditnote

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Credit note UUID
original_invoice_id	UUID	FK Bill	Original invoice being credited
return_request_id	BIGINT	FK ReturnRequest, NULL	Associated return request
credit_note_number	VARCHAR(50)	UNIQUE	CRN-YYYY-XXX sequential format
taxable_reversal	FLOAT	DEFAULT 0	Taxable value being reversed
gst_reversal	FLOAT	DEFAULT 0	Total GST amount reversed
cgst_reversal	FLOAT	DEFAULT 0	CGST component reversed
sgst_reversal	FLOAT	DEFAULT 0	SGST component reversed
created_at	DATETIME	auto_now_add	Credit note creation timestamp

3.3.7 Table: payments_paymenttransaction

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Transaction UUID
bill_id	UUID	FK Bill, OneToOne	Associated bill (one transaction per bill)
payment_mode	VARCHAR(20)	NOT NULL	UPI / CARD / CASH / CREDIT
amount	FLOAT	NOT NULL	Transaction amount in Rs.
transaction_id	VARCHAR(255)	NULL	External gateway reference ID
status	VARCHAR(15)	DEFAULT PENDING	PENDING / PAID / FAILED
is_confirmed_by_seller	BOOLEAN	DEFAULT FALSE	Seller has confirmed receipt
is_confirmed_by_customer	BOOLEAN	DEFAULT FALSE	Buyer has confirmed payment
created_at	DATETIME	auto_now_add	Transaction creation timestamp

3.3.8 Table: billing_gstfiling

Column	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, auto	Filing record ID
seller_id	UUID	FK CustomUser	GST seller this period belongs to
month_year	VARCHAR(20)	NOT NULL	Filing period e.g. 2026-04
total_sales	FLOAT	DEFAULT 0	Total invoiced sales for the period
cgst	FLOAT	DEFAULT 0	Total CGST collected in the period
sgst	FLOAT	DEFAULT 0	Total SGST collected in the period
bill_count	INT	DEFAULT 0	Number of PAID invoices in the period

3.4 Data Flow Diagrams / Activity Diagrams

4.4.1 Level 0 DFD (Context Diagram)

The context diagram treats the entire system as a single process and shows all external entities and the major data flows into and out of the system. External entities include the GST Seller, Non-GST Customer, System Administrator, and the SMTP Email Service.

Level 0 DFD — Context Diagram

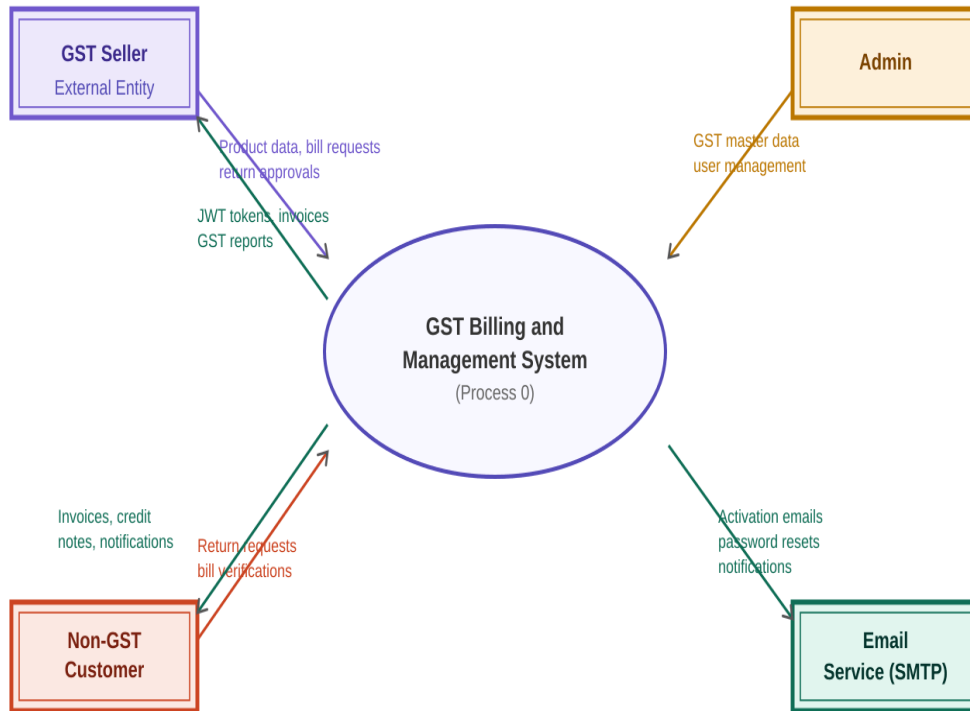


Figure 3.3: Level 0 DFD — Context Diagram

4.4.2 Level 1 DFD (Main Processes)

The Level 1 DFD decomposes the system into five major processes with their data stores and flows. Process 1.0 handles all user management, Process 2.0 handles billing, Process 3.0 handles returns, Process 4.0 manages the wallet and referral system, and Process 5.0 generates GST filing summaries.

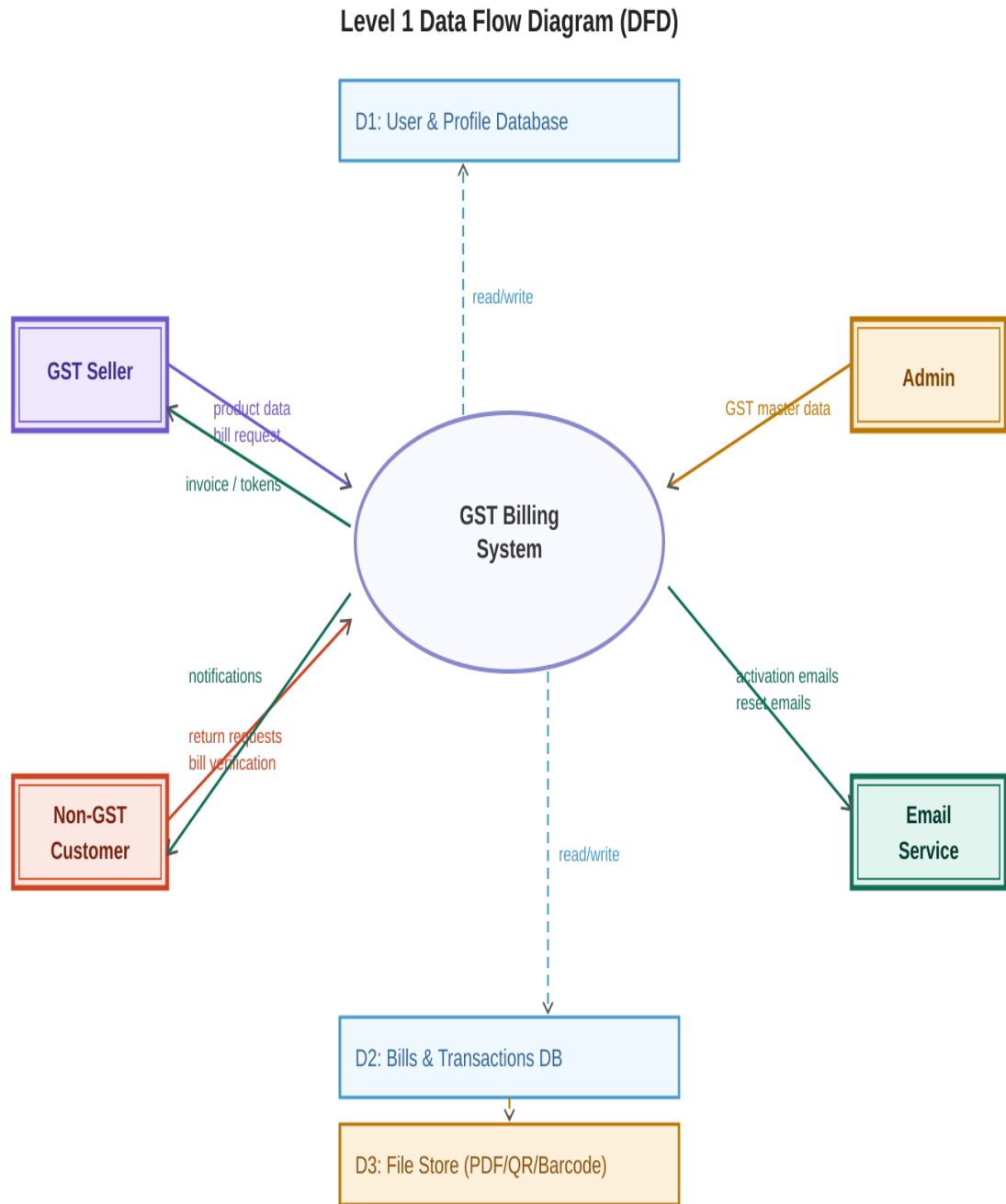


Figure 3.4: Level 1 DFD — Main System Processes

4.4.3 Bill Creation Activity Flow

The following describes the step-by-step data flow during bill creation:

40. Seller submits POST `/api/billing/create-bill/` with customer Bill ID, items list, payment mode, and optional discount.
41. System verifies the seller's GST profile is both completed and verified. If not, a 403 Forbidden response is returned.

42. System fetches the customer by Bill ID. If seller equals customer, a 400 Bad Request is returned.
43. For each item: product is fetched, Taxable Value = Qty x Price, GST Amount = Taxable x (GST%/100), CGST = GST/2, SGST = GST/2. GST rate is validated against master (VALID, WARNING, or INVALID).
44. Discount is applied: Discount Amount = Total x Discount%. Final Total = Total minus Discount Amount.
45. Bill and all BillItem records are saved. Invoice number INV-XXXX is auto-assigned.
46. For UPI or CREDIT payment mode, a verification notification is sent to the customer.
47. Invoice number, computed totals, and gst_status are returned to the seller.

3.5 Flowcharts and Sequence Diagrams

3.5.1 User Registration and Authentication Flowchart

The flowchart below shows the complete user registration process including field validation, duplicate checking, wallet creation for Non-GST users, email token generation, and account activation.

User Registration and Authentication Flowchart

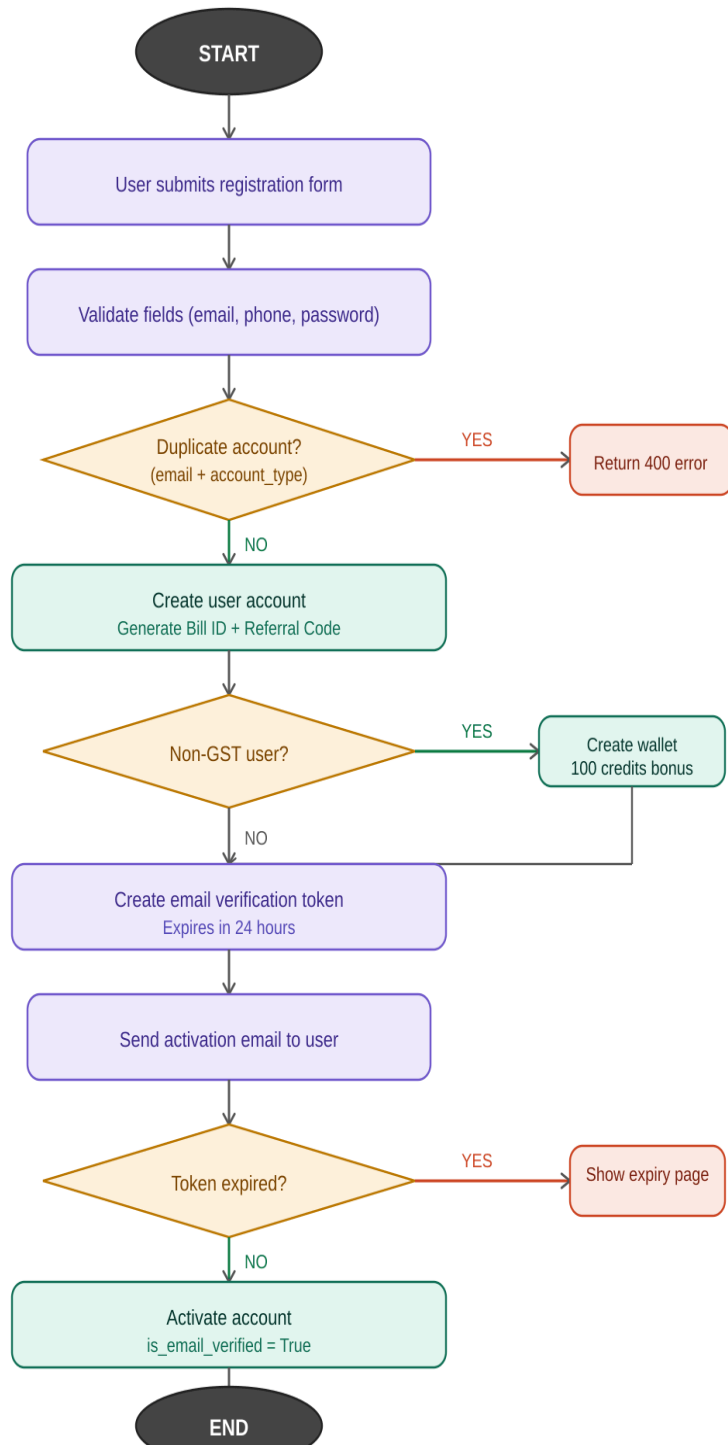


Figure 3.5: User Registration and Authentication Flowchart

3.5.2 Bill Creation Activity Flowchart

The flowchart below shows the complete bill creation process from GST profile validation through item-level CGST/SGST computation, GST master validation, discount application, and invoice generation.

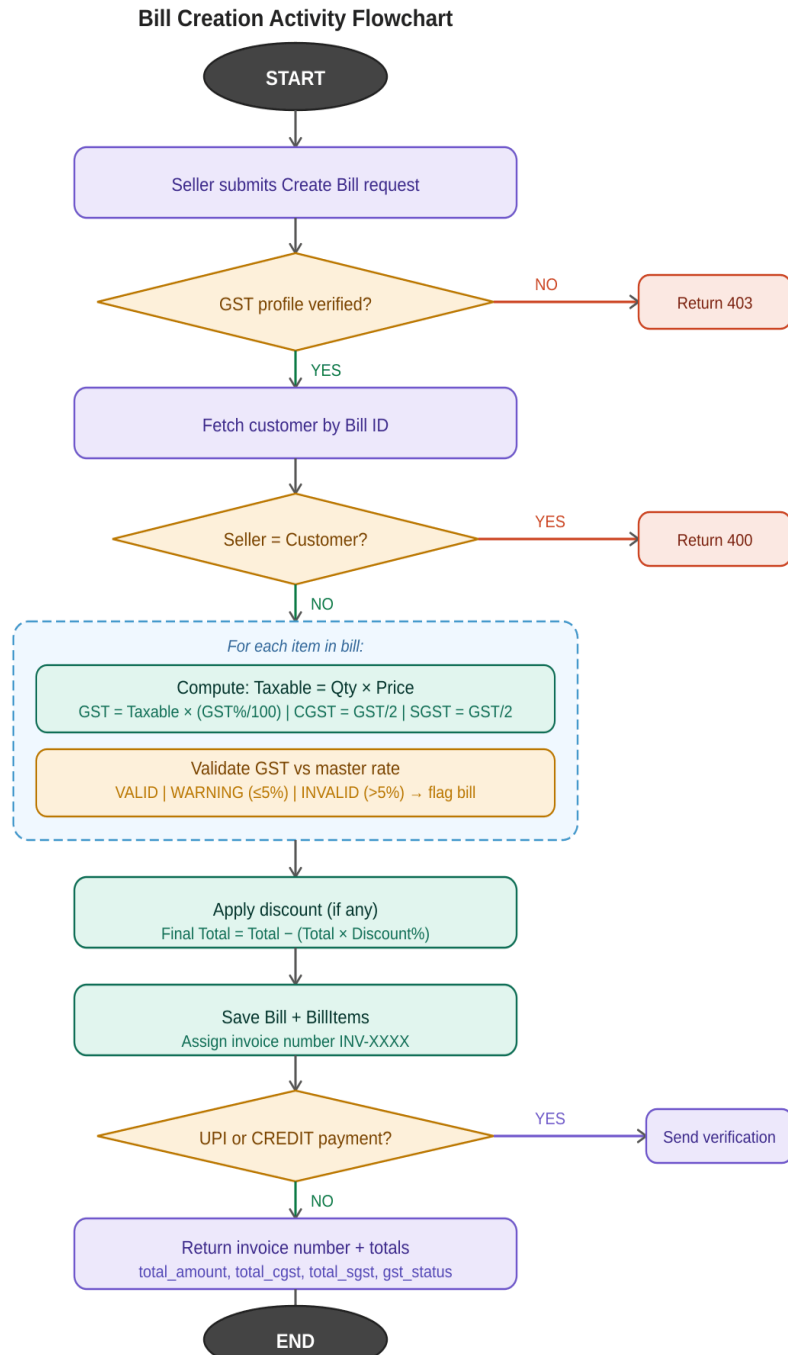


Figure 3.6: Bill Creation Activity Flowchart with GST Computation

3.5.3 Bill Payment Sequence Diagram (UPI Mode)

The sequence diagram below shows the message flow between the Seller, System API, Database, and Customer during UPI bill creation, verification, and payment confirmation. Solid arrows represent requests; dashed arrows represent responses.

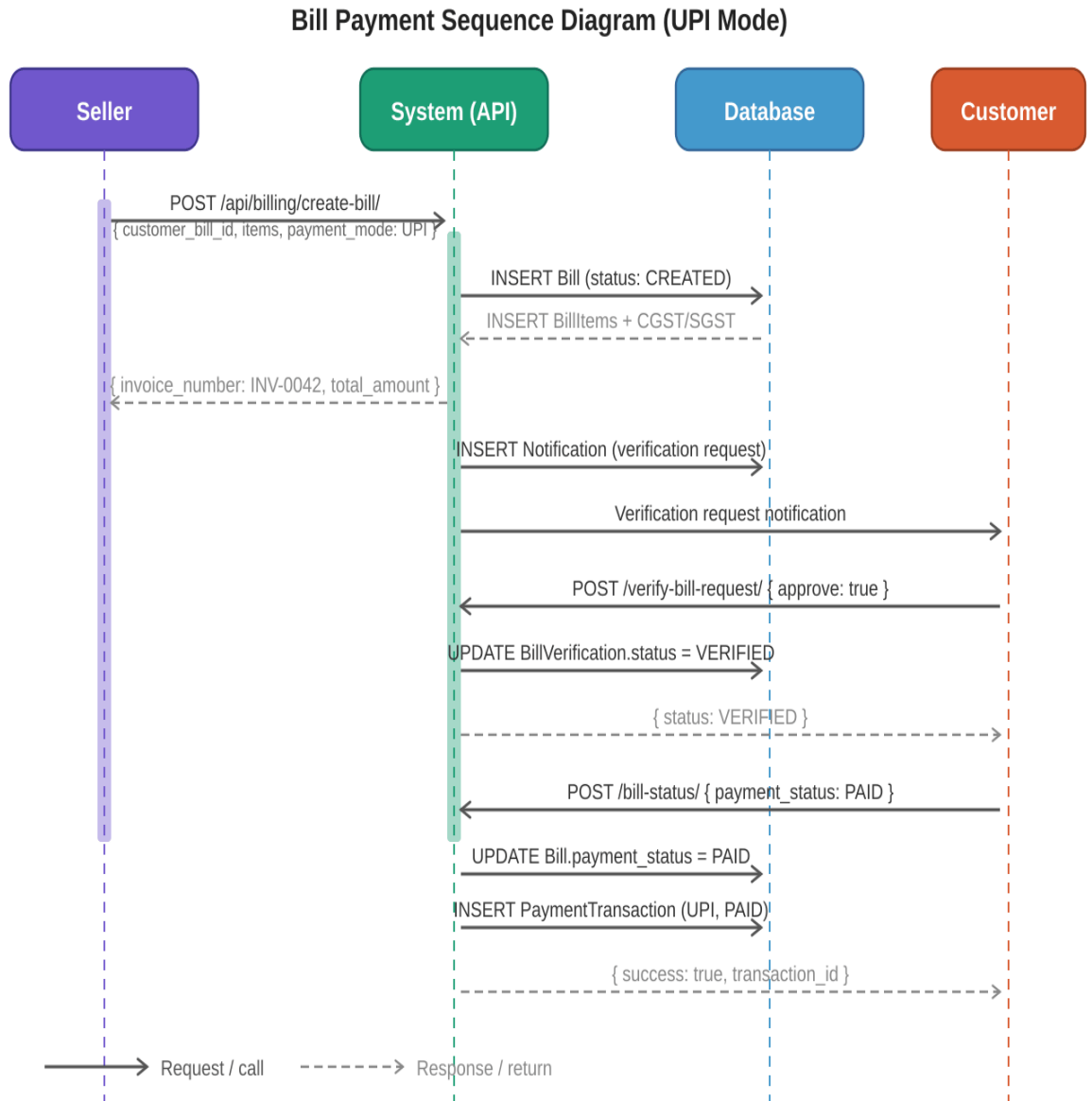


Figure 3.7: Bill Payment Sequence Diagram — UPI Mode

3.5.4 Return Processing Flowchart

The flowchart below shows the complete return request lifecycle from customer submission through seller approval, proportional GST reversal computation, credit note generation, and bill status update.

Return and Exchange Processing Flowchart

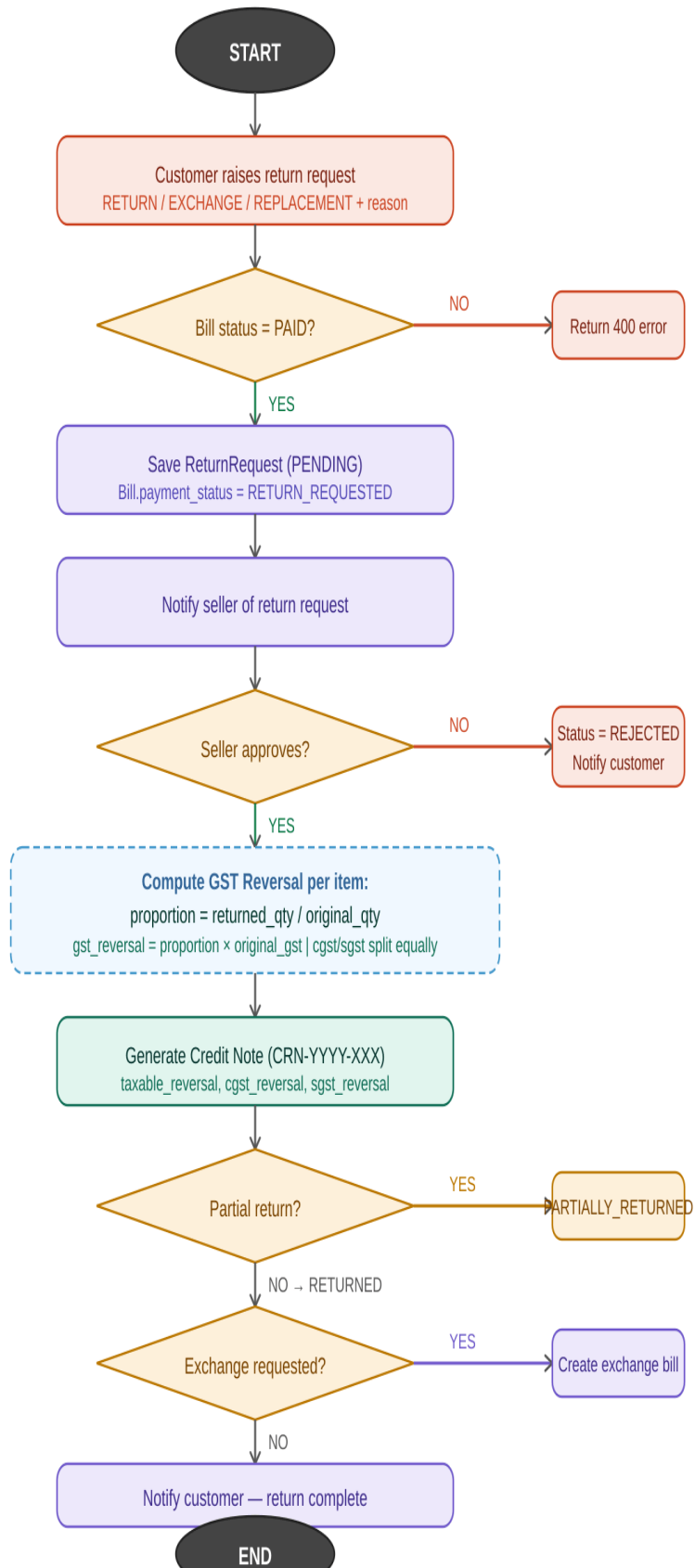


Figure 3.8: Return and Exchange Processing Flowchart

4.6 Module Interaction Diagram

The three Django applications interact with each other through well-defined interfaces. The accounts app provides the user authentication context that all other apps depend on. The billing app references accounts for seller and customer user objects and writes to its own models. The payments app references billing's Bill model to create payment transactions.

Source Module	Depends On	Nature of Dependency
billing.views	accounts.models.CustomUser	Foreign key: Bill.seller, Bill.customer
billing.models.Bill	billing.models.BillItem	One-to-many: Bill aggregates BillItems
billing.models.ReturnRequest	billing.models.Bill	Many-to-one: Return references original Bill
billing.models.CreditNote	billing.models.ReturnRequest	Many-to-one: Credit note references return
billing.views.CreateBillAPI	billing.utils.validate_gst	Function call for GST rate validation
billing.views.CreateBillAPI	accounts.models.Wallet	Direct model access for CREDIT payments
payments.models.PaymentTransaction	billing.models.Bill	One-to-one: Payment linked to Bill
accounts.views.RegisterAPI	accounts.models.Wallet	Creates wallet on Non-GST registration
accounts.views.RegisterAPI	accounts.models.CreditTransaction	Records signup bonus transaction

4.7 Security Design

The system implements multiple layers of security to protect financial and personal data:

4.7.1 Authentication Security

- JWT (JSON Web Token) authentication is used for all protected endpoints. Access tokens have a short expiry (typically 60 minutes) while refresh tokens have a longer expiry (24 hours).
- On logout, the refresh token is blacklisted using SimpleJWT's token blacklist mechanism, preventing token reuse even if the token has not yet expired.
- Password storage uses Django's PBKDF2 algorithm with SHA-256 hashing and a random per-user salt. Raw passwords are never stored or logged.
- Email verification tokens use Django's uuid4 for generation, ensuring cryptographic randomness. Tokens expire after 24 hours and are deleted upon use.

4.7.2 Authorization Security

- Every API endpoint checks that the authenticated user has the correct `account_type` (GST or Non-GST) before executing business logic.
- Product management endpoints verify that the product belongs to the requesting seller before allowing update or delete operations.
- Bill detail and export endpoints verify that the requesting user is either the seller or customer of that specific bill.
- Return request approval and completion endpoints verify that the requesting user is the seller assigned to that return request.

4.7.3 Data Integrity Security

- Critical bill fields are locked after payment using a custom `save()` override that raises an exception on any attempt to modify `total_amount`, `seller`, or `customer` of a PAID bill.
- Invoice numbers are generated with database-level UNIQUE constraints, preventing duplicate invoice numbers even under concurrent request conditions.
- All database queries use Django ORM's parameterized queries, preventing SQL injection attacks.
- CORS (Cross-Origin Resource Sharing) is configured via `django-cors-headers` to restrict which frontend origins can access the API.

4.8 Error Handling Design

The system follows a consistent error response format across all endpoints to make error handling predictable for frontend clients:

```
# Standard error response format:
{
  'success': false,
  'error': 'Human-readable error message',
  'code': 'ERROR_CODE'    // optional machine-readable code
}

# Standard success response format:
{
  'success': true,
  'message': 'Operation completed successfully',
  'data': { ... }    // response payload
}
```

HTTP Status Code	Usage in This System
200 OK	Successful GET, PUT requests
201 Created	Successful POST that creates a new resource
400 Bad Request	Validation errors, business rule violations (self-billing, insufficient credits)
401 Unauthorized	Missing or invalid JWT token
403 Forbidden	Valid token but insufficient permissions (wrong account type, not owner)
404 Not Found	Requested resource does not exist
500 Internal Server Error	Unexpected server-side error (logged for debugging)

4.9 Deployment Architecture

The recommended production deployment architecture for the GST Billing and Management System is as follows:

Component	Technology / Configuration
Web Server	Nginx — acts as reverse proxy, serves static and media files, handles SSL termination
Application Server	Gunicorn — WSGI server running Django application, 4 worker processes
Database	PostgreSQL 14+ with connection pooling via pgBouncer
Cache	Redis — for Django session cache and JWT token blacklist storage
File Storage	Local filesystem (development) or AWS S3 (production) for media files
Email	SMTP via SendGrid or AWS SES for transactional emails
Process Manager	Supervisor or systemd for managing Gunicorn processes
SSL Certificate	Let's Encrypt (free) or paid SSL certificate on Nginx
Environment Variables	Python-decouple or django-environ for secrets management
Monitoring	Django logging to file + Sentry for error tracking (optional)

CHAPTER 4: CODING — MAIN MODULES

The project is built using Django 4.2 and Django REST Framework, organized into three modular Django applications. The codebase follows Django's MTV (Model-Template-View) pattern with REST API views replacing traditional templates. Below are the key modules with their complete implementation logic.

5.1 Custom User Model (accounts/models.py)

The system uses a custom `AbstractBaseUser` model extending Django's default user to support UUID primary keys, dual account types, auto-generated Bill IDs, referral codes, QR codes, and barcodes.

```
class CustomUser(AbstractBaseUser, PermissionsMixin):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4,
editable=False)
    email = models.EmailField(unique=False)
    name = models.CharField(max_length=200)
    phone = models.CharField(max_length=15)
    account_type = models.CharField(
        max_length=20, choices=[('GST', 'GST'), ('Non-GST', 'Non-GST')]
    )
    bill_id = models.CharField(max_length=100, unique=True, blank=True)
    my_referral_code = models.CharField(max_length=20, unique=True,
        blank=True, null=True)
    referred_by = models.ForeignKey('self', on_delete=models.SET_NULL,
        null=True, blank=True, related_name='referrals')
    gstin = models.CharField(max_length=20, blank=True, null=True)
    gst_verified = models.BooleanField(default=False)
    profile_completed = models.BooleanField(default=False)
    qr_code = models.ImageField(upload_to='qr_codes/', blank=True,
null=True)
    barcode = models.ImageField(upload_to='barcodes/', blank=True,
null=True)

    class Meta:
        unique_together = ('email', 'account_type')

    def save(self, *args, **kwargs):
        if not self.bill_id:
            self.bill_id = self.generate_bill_id()
        if not self.my_referral_code:
            self.my_referral_code = self.generate_referral_code()
        super().save(*args, **kwargs)
        if self.account_type == 'Non-GST':
            if not self.qr_code: # auto-generate QR code image
                qr = qrcode.make(self.bill_id) # saves to
media/qr_codes/
                if not self.barcode: # auto-generate barcode image
                    code = barcode.get('code128', self.bill_id,
writer=ImageWriter())
```

5.2 User Registration API (accounts/views.py)

```
class RegisterAPI(APIView):
    def post(self, request):
        serializer = RegisterSerializer(data=request.data)
        if serializer.is_valid():
            user = serializer.save()

            # Credit 100 bonus points to new Non-GST users
            if user.account_type == 'Non-GST':
                Wallet.objects.create(user=user, balance=100)
                CreditTransaction.objects.create(
                    user=user, amount=100, type='CREDIT',
                    reason='Signup Bonus'
                )

            # Generate email verification token (expires 24h)
            token_obj = EmailVerificationToken.objects.create(user=user)
            send_activation_email(
                user.email, token_obj.token, user.account_type
            )

            return Response({
                'success': True,
                'message': 'User registered. Check email for activation
link.',
                'data': {
                    'bill_id': user.bill_id,
                    'referral_code': user.my_referral_code,
                }
            }, status=201)

        return Response({'success': False, 'data': serializer.errors},
                        status=400)
```

5.3 GST Profile Verification Guard (billing/views.py)

This utility function is called at the entry of every billing API endpoint. It ensures only GST sellers with verified and completed business profiles can create products or bills.

```
def require_verified_gst_profile(user):
    if user.account_type != 'GST':
        return Response(
            {'error': 'Only GST users can perform this action'},
            status=403
        )
    if not user.gst_verified or not user.profile_completed:
        return Response({
            'error': 'Complete and verify your GST profile before
adding products or processing bills'
        }, status=403)
    return None # profile is valid, allow the request to proceed
```

5.4 GST Validation Utility (billing/utls.py)

This function validates the applied GST rate against the registered product rate from the master database and returns the compliance status for each bill item.

```
def validate_gst(product_gst, applied_gst):  
    '''  
    Validates applied GST rate against registered product rate.  
    Returns:  
        VALID - exact match or zero difference  
        WARNING - difference is > 0 and <= 5%  
        INVALID - difference exceeds 5% (bill will be flagged)  
    '''  
    difference = abs(product_gst - applied_gst)  
    if difference == 0:  
        return 'VALID'  
    elif difference <= 5:  
        return 'WARNING'  
    else:  
        return 'INVALID'
```

5.5 Bill Creation with CGST/SGST Computation (billing/views.py)

This is the core billing module. It creates a bill with item-level CGST and SGST computation, applies GST validation against master data, handles discounts, manages bill verification for UPI/CREDIT payments, and handles CREDIT mode payment deduction.

```
class CreateBillAPI(APIView):  
    permission_classes = [IsAuthenticated]  
  
    def post(self, request):  
        profile_error = require_verified_gst_profile(request.user)  
        if profile_error: return profile_error  
  
        customer = get_object_or_404(  
            User, bill_id=request.data.get('bill_id')  
        )  
        if customer == request.user:  
            return Response(  
                {'error': 'Seller cannot create a bill for themselves'},  
                status=400  
            )  
  
        bill = Bill.objects.create(seller=request.user,  
customer=customer)  
        total_amount = total_gst = total_cgst = 0  
        total_sgst = total_taxable = 0  
        final_gst_status = 'VALID'  
        is_flagged = False  
  
        for item in request.data.get('items', []):  
            product = get_object_or_404(  
                Product, id=item['product_id'], seller=request.user  
            )  
            qty = int(item.get('quantity', 1))  
            price = float(item.get('price', product.price))  
            gst_pct = float(item.get('gst_percentage',  
product.gst_percentage))
```

```

# Core GST Computation per item
taxable_value = round(qty * price, 2)
gst_amount    = round(taxable_value * (gst_pct / 100), 2)
cgst          = round(gst_amount / 2, 2)
sgst          = round(gst_amount / 2, 2)
total_price   = round(taxable_value + gst_amount, 2)

# Validate GST rate against master
status = validate_gst(product.gst_percentage, gst_pct)
if status == 'INVALID':
    is_flagged = True
    final_gst_status = 'INVALID'
elif status == 'WARNING' and final_gst_status != 'INVALID':
    final_gst_status = 'WARNING'

BillItem.objects.create(
    bill=bill, product=product, name=product.name,
    quantity=qty, price=price, gst_percentage=gst_pct,
    taxable_value=taxable_value, gst_amount=gst_amount,
    cgst=cgst, sgst=sgst, total_price=total_price
)
total_taxable += taxable_value
total_gst     += gst_amount
total_cgst    += cgst
total_sgst    += sgst
total_amount  += total_price

# Apply bill-level discount
discount_pct = float(request.data.get('discount', 0))
discount_amt = round(total_amount * discount_pct / 100, 2)
final_total  = round(total_amount - discount_amt, 2)

bill.total_amount      = final_total
bill.total_gst         = round(total_gst, 2)
bill.total_taxable_value = round(total_taxable, 2)
bill.total_cgst        = round(total_cgst, 2)
bill.total_sgst        = round(total_sgst, 2)
bill.gst_status        = final_gst_status
bill.is_flagged        = is_flagged
bill.payment_mode      = request.data.get('payment_mode',
'CASH')
bill.discount_percentage = discount_pct
bill.discount_amount     = discount_amt
bill.save()

return Response({
    'invoice_number': bill.invoice_number,
    'total_amount'  : bill.total_amount,
    'total_cgst'    : bill.total_cgst,
    'total_sgst'    : bill.total_sgst,
    'gst_status'    : bill.gst_status,
    'discount_amount': discount_amt
}, status=201)

```

5.6 Bill Model with Auto Invoice Numbering (billing/models.py)

The Bill model includes a custom save() method that auto-generates sequential invoice numbers and locks critical financial fields once a bill reaches PAID status.

```
class Bill(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4,
                           editable=False)
    seller = models.ForeignKey(User, related_name='seller_bills', ...)
    customer = models.ForeignKey(User, related_name='customer_bills',
    ...)
    invoice_number = models.CharField(max_length=20, unique=True,
blank=True)
    total_amount = models.FloatField(default=0)
    total_cgst = models.FloatField(default=0)
    total_sgst = models.FloatField(default=0)
    payment_status = models.CharField(max_length=20, default='CREATED')
    gst_status = models.CharField(max_length=10, default='PENDING')
    is_flagged = models.BooleanField(default=False)

    def save(self, *args, **kwargs):
        # Lock critical fields once bill is paid
        if self.pk:
            try:
                old = Bill.objects.get(pk=self.pk)
                if old.payment_status in ['PAID', 'RETURNED', 'EXCHANGED']:
                    if old.total_amount != self.total_amount:
                        raise Exception(
                            'Critical financial fields locked after
payment'
                        )
            except Bill.DoesNotExist:
                pass

        # Auto-generate sequential invoice number
        if not self.invoice_number:
            last = Bill.objects.order_by('-created_at').first()
            if last and last.invoice_number:
                n = int(last.invoice_number.split('-')[-1]) + 1
            else:
                n = 1
            self.invoice_number = f'INV-{str(n).zfill(4)}'

        super().save(*args, **kwargs)
```

5.7 PDF Invoice Export (billing/views.py)

This module uses the ReportLab library to generate a GST-compliant PDF invoice containing seller GSTIN details, buyer information, itemized table with CGST and SGST columns, discount summary, and grand total.

```
class ExportBillPDFAPI(APIView):
    def get(self, request):
        bill = get_object_or_404(Bill, id=request.GET.get('bill_id'))
        if request.user not in [bill.seller, bill.customer]:
            return Response({'error': 'Access denied'}, status=403)

        buffer = io.BytesIO()
        p = canvas.Canvas(buffer, pagesize=letter)
```


5.8 Return Completion and Credit Note Generation (billing/views.py)

When the seller marks a return as complete, this module computes proportional GST reversal for each returned item and generates a credit note with individual CGST and SGST reversal amounts.

```
class CompleteReturnAPI(APIView):
    def post(self, request):
        rr = get_object_or_404(
            ReturnRequest, id=request.data.get('return_id'),
            seller=request.user
        )
        taxable_rev = gst_rev = cgst_rev = sgst_rev = 0

        # Compute proportional GST reversal for each returned item
        for ret_item in rr.items.all():
            bi = ret_item.bill_item
            proportion = ret_item.quantity / bi.quantity
            taxable_rev += round(bi.taxable_value * proportion, 2)
            gst_rev += round(bi.gst_amount * proportion, 2)
            cgst_rev += round(bi.cgst * proportion, 2)
            sgst_rev += round(bi.sgst * proportion, 2)

        # Create Credit Note (auto-assigned CRN-YYYY-XXX number)
        cn = CreditNote.objects.create(
            original_invoice = rr.original_bill,
            return_request = rr,
            taxable_reversal = round(taxable_rev, 2),
            gst_reversal = round(gst_rev, 2),
            cgst_reversal = round(cgst_rev, 2),
            sgst_reversal = round(sgst_rev, 2)
        )

        rr.status = 'COMPLETED'
        rr.gst_reversal_amount = gst_rev
        rr.save()

        # Update original bill status
        if rr.is_partial_return:
            rr.original_bill.payment_status = 'PARTIALLY_RETURNED'
        else:
            rr.original_bill.payment_status = 'RETURNED'
        rr.original_bill.save()

        return Response({
            'credit_note_number': cn.credit_note_number,
            'gst_reversal': cn.gst_reversal,
            'cgst_reversal': cn.cgst_reversal,
            'sgst_reversal': cn.sgst_reversal
        }, status=200)
```

5.9 GST Filing Monthly Summary (billing/views.py)

```
class GSTFilingAPI(APIView):
    def get(self, request):
        profile_error = require_verified_gst_profile(request.user)
        if profile_error: return profile_error

        filings = GSTFiling.objects.filter(
            seller=request.user
        ).order_by('-month_year')

        data = [
            {
                'month_year' : f.month_year,
                'total_sales' : f.total_sales,
                'cgst'       : f.cgst,
                'sgst'       : f.sgst,
                'bill_count'  : f.bill_count
            }
            for f in filings
        ]
        return Response(data, status=200)
```

5.10 Product Management API (billing/views.py)

The product management module allows verified GST sellers to add new products, update existing ones by barcode, and delete or list their product catalog. It enforces ownership — a seller can only manage their own products.

```
class SelectProductsAPI(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        profile_error = require_verified_gst_profile(request.user)
        if profile_error: return profile_error

        barcode    = request.data.get('barcode')
        name        = request.data.get('name')
        price       = float(request.data.get('price', 0))
        gst_pct     = float(request.data.get('gst_percentage', 0))
        hsn_code    = request.data.get('hsn_code', '')
        brand_name  = request.data.get('brand_name', '')

        # Create new product or update existing by barcode
        product, created = Product.objects.update_or_create(
            seller=request.user,
            barcode=barcode,
            defaults={
                'name'           : name,
                'price'          : price,
                'gst_percentage' : gst_pct,
                'hsn_code'       : hsn_code,
                'brand_name'     : brand_name,
            }
        )
        return Response({
            'product_id': product.id,
            'created'    : created,
            'message'    : 'Product added' if created else 'Product
updated'
        }, status=201 if created else 200)
```

```

class ProductByBarcodeAPI(APIView):
    def get(self, request):
        barcode = request.GET.get('barcode')
        product = get_object_or_404(
            Product, barcode=barcode, seller=request.user
        )
        return Response({
            'product_id' : product.id,
            'name'       : product.name,
            'brand_name' : product.brand_name,
            'price'      : product.price,
            'gst_percentage': product.gst_percentage,
            'hsn_code'   : product.hsn_code,
        }, status=200)

```

5.11 Bill Status Update and Payment Confirmation (billing/views.py)

This API allows the seller or customer to update the bill's payment status. It also handles CREDIT payment deduction from the customer's wallet when the payment mode is CREDIT.

```

class BillStatusAPI(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        bill_id = request.data.get('bill_id')
        payment_status = request.data.get('payment_status')
        bill = get_object_or_404(Bill, id=bill_id)

        # Only seller or customer can update status
        if request.user not in [bill.seller, bill.customer]:
            return Response({'error': 'Access denied'}, status=403)

        # Handle CREDIT payment deduction
        if (payment_status == 'PAID' and
            bill.payment_mode == 'CREDIT'):
            wallet = get_object_or_404(Wallet, user=bill.customer)
            credits_needed = int(math.ceil(bill.total_amount))
            if wallet.balance < credits_needed:
                return Response({
                    'error': f'Insufficient credits. Need
{credits_needed}, '
                                f' have {wallet.balance}'
                }, status=400)
            wallet.balance -= credits_needed
            wallet.save()
            CreditTransaction.objects.create(
                user=bill.customer, amount=credits_needed,
                type='DEBIT',
                reason=f'Payment for {bill.invoice_number}'
            )

        bill.payment_status = payment_status
        bill.save()

        # Create PaymentTransaction record
        if payment_status == 'PAID':

```

```

        PaymentTransaction.objects.get_or_create(
            bill=bill,
            defaults={
                'payment_mode': bill.payment_mode,
                'amount': bill.total_amount,
                'status': 'PAID'
            }
        )

    return Response({
        'success': True,
        'bill_id': str(bill.id),
        'payment_status': bill.payment_status
    }, status=200)

```

5.12 Seller Dashboard API (billing/views.py)

The dashboard API aggregates key business metrics for the GST seller, including total sales, GST collected, pending bills, flagged bills, and a summary for the current month. This data is intended for display on a seller analytics dashboard.

```

class DashboardAPI(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        profile_error = require_verified_gst_profile(request.user)
        if profile_error: return profile_error

        seller = request.user
        all_bills = Bill.objects.filter(
            seller=seller, payment_status='PAID'
        )

        # Aggregate totals across all paid bills
        total_sales = sum(b.total_amount for b in all_bills)
        total_cgst = sum(b.total_cgst for b in all_bills)
        total_sgst = sum(b.total_sgst for b in all_bills)

        # Current month stats
        now = datetime.now()
        month_bills = all_bills.filter(
            created_at__year=now.year,
            created_at__month=now.month
        )
        month_sales = sum(b.total_amount for b in month_bills)
        month_cgst = sum(b.total_cgst for b in month_bills)

        # Pending and flagged bills
        pending = Bill.objects.filter(
            seller=seller, payment_status='CREATED'
        ).count()
        flagged = Bill.objects.filter(
            seller=seller, is_flagged=True
        ).count()

        # 5 most recent bills
        recent = Bill.objects.filter(
            seller=seller
        ).order_by('-created_at')[:5]

```

```

return Response({
    'total_sales'          : round(total_sales, 2),
    'total_cgst_collected' : round(total_cgst, 2),
    'total_sgst_collected' : round(total_sgst, 2),
    'total_bills'          : all_bills.count(),
    'pending_bills'        : pending,
    'flagged_bills'        : flagged,
    'this_month': {
        'sales'            : round(month_sales, 2),
        'cgst'             : round(month_cgst, 2),
        'sgst'             : round(month_sgst, 2),
        'bill_count'       : month_bills.count()
    },
    'recent_bills': [
        {
            'invoice_number': b.invoice_number,
            'customer'      : b.customer.name,
            'amount'        : b.total_amount,
            'status'        : b.payment_status,
            'date'          : b.created_at.strftime('%d-%m-%Y')
        }
        for b in recent
    ]
}, status=200)

```

5.13 Serializers (accounts/serializers.py and billing/serializers.py)

Serializers perform input validation and output formatting. The RegisterSerializer validates all registration fields and enforces the unique (email, account_type) constraint. The BillItemSerializer validates individual item fields during bill creation.

```

# accounts/serializers.py
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, min_length=8)

    class Meta:
        model = CustomUser
        fields = ['name', 'email', 'phone', 'account_type', 'password',
                  'referral_code']

    def validate(self, data):
        # Enforce unique (email, account_type) constraint
        if CustomUser.objects.filter(
            email=data['email'],
            account_type=data['account_type']
        ).exists():
            raise serializers.ValidationError(
                'Account with this email and type already exists.'
            )
        return data

    def create(self, validated_data):
        referral_code = validated_data.pop('referral_code', None)
        password = validated_data.pop('password')
        user = CustomUser(**validated_data)
        user.set_password(password) # hashes password via PBKDF2
        if referral_code:

```

```

        try:
            referrer = CustomUser.objects.get(
                my_referral_code=referral_code
            )
            user.referred_by = referrer
        except CustomUser.DoesNotExist:
            pass # invalid code; ignore silently
    user.save()
    return user

# billing/serializers.py
class BillItemInputSerializer(serializers.Serializer):
    product_id      = serializers.IntegerField()
    quantity        = serializers.IntegerField(min_value=1)
    price           = serializers.FloatField(min_value=0)
    gst_percentage  = serializers.FloatField(min_value=0, max_value=100)

```

CHAPTER 5: TESTING

5.1 Test Cases

The GST Billing and Management System was tested using three complementary approaches. Unit testing covered individual API endpoints in isolation to verify correct responses for both valid and invalid inputs. Integration testing covered end-to-end workflows spanning multiple APIs, such as registration → activation → login → product creation → bill creation → payment → PDF export. Boundary testing covered edge cases including zero quantities, self-billing attempts, expired tokens, and bills being modified after payment. Postman was the primary tool for executing and documenting all test cases.

5.2 Test Cases — User Registration and Authentication

TC ID	Test Case Description	Expected Result	Status
TC-001	Register new GST user with all valid fields	201 Created; activation email sent; bill_id and referral_code returned	PASS
TC-002	Register with same email and account_type (duplicate check)	400 Bad Request; duplicate account error message returned	PASS
TC-003	Register new Non-GST user	201 Created; wallet created with 100 credits; CreditTransaction recorded	PASS
TC-004	Activate account with valid token within 24 hours	Account activated; is_email_verified = True; login becomes possible	PASS
TC-005	Activate account with expired token (> 24 hours old)	Activation expired page displayed; account remains inactive	PASS
TC-006	Login with correct email and password	200 OK; JWT access token and refresh token returned	PASS
TC-007	Login with correct email but wrong password	401 Unauthorized; authentication failed message	PASS
TC-008	Access protected API endpoint without JWT token	401 Unauthorized; authentication credentials not provided	PASS
TC-009	Request password reset for valid registered email	200 OK; reset email dispatched; token created with 1 hour expiry	PASS
TC-010	Reset password using expired token (> 1 hour)	400 Bad Request; token expired message	PASS
TC-011	Login with correct credentials but unactivated account	400 Bad Request; account not yet activated message	PASS
TC-012	Resend activation email for unactivated account	200 OK; new activation token created; email sent successfully	PASS

5.3 Test Cases — Product Management

TC ID	Test Case Description	Expected Result	Status
TC-013	Add product by GST seller with <code>gst_verified = False</code>	403 Forbidden; GST profile not verified error	PASS
TC-014	Add product by verified GST seller with all required fields	201 Created; product saved with <code>id</code> , <code>barcode</code> , <code>hsn_code</code>	PASS
TC-015	Add product with barcode that already exists for same seller	200 OK; existing product updated with new data	PASS
TC-016	Fetch product by valid barcode belonging to authenticated seller	200 OK; full product details returned	PASS
TC-017	Fetch product using barcode belonging to a different seller	404 Not Found; product not found for this seller	PASS
TC-018	Update product price via <code>update-product</code> endpoint	200 OK; product price updated successfully	PASS
TC-019	Delete product via <code>delete-product</code> endpoint	200 OK; product deleted; no longer fetchable	PASS

5.4 Test Cases — Billing and GST Computation

TC ID	Test Case Description	Expected Result	Status
TC-020	Create bill with valid items and CASH payment mode	201 Created; <code>invoice_number</code> assigned; CGST/SGST computed correctly	PASS
TC-021	Create bill where seller Bill ID = customer Bill ID	400 Bad Request; seller cannot create bill for themselves	PASS
TC-022	Create bill with an item having quantity = 0	400 Bad Request; quantity must be greater than zero	PASS
TC-023	Create bill with GST rate 3% above product's registered rate	201 Created; <code>gst_status</code> = WARNING; <code>is_flagged</code> = False	PASS
TC-024	Create bill with GST rate 10% above product's registered rate	201 Created; <code>gst_status</code> = INVALID; <code>is_flagged</code> = True	PASS
TC-025	Create bill with 10% discount applied	201 Created; <code>discount_amount</code> = total x 10%; grand total reduced correctly	PASS
TC-026	Export bill as PDF for authenticated seller	200 OK; PDF binary response; Content-Disposition header present	PASS
TC-027	Export bill as Excel for authenticated customer	200 OK; Excel binary response with all item columns	PASS
TC-028	Access bill detail for a user not linked to the bill	403 Forbidden; access denied	PASS
TC-029	Attempt to modify <code>total_amount</code> of a PAID bill	Exception raised; modification blocked; fields locked after payment	PASS
TC-030	CGST/SGST accuracy: price=1000, qty=2, GST=18%	<code>taxable</code> =2000; <code>gst</code> =360; <code>cgst</code> =180; <code>sgst</code> =180; <code>total</code> =2360	PASS
TC-031	Bill with 5% discount: total=2360, discount=5%	<code>discount_amount</code> =118; <code>final_total</code> =2242.00	PASS

5.5 Test Cases — Return, Exchange, and Credit Note

TC ID	Test Case Description	Expected Result	Status
TC-032	Raise return request on a bill with status CREATED (unpaid)	400 Bad Request; only PAID bills can have return requests	PASS
TC-033	Raise valid return request on PAID bill with item selection	201 Created; ReturnRequest created; bill status = RETURN_REQUESTED	PASS
TC-034	Seller approves pending return request	200 OK; return status updated to APPROVED	PASS
TC-035	Complete return for all items with full quantities	Bill status = RETURNED; CreditNote created with full GST reversal	PASS
TC-036	Complete return for one item with partial quantity (1 of 2)	Bill status = PARTIALLY_RETURNED; CreditNote with 50% GST reversal	PASS
TC-037	Seller rejects return request	Return status = REJECTED; bill status restored to PAID; customer notified	PASS
TC-038	Credit note number format check	Credit note number follows CRN-YYYY-XXX format correctly	PASS
TC-039	GST reversal accuracy: original qty=2, returned qty=1, gst=360	GST reversal = 180 (50% of 360); cgst_rev=90; sgst_rev=90	PASS
TC-040	Create exchange bill after return approval	New Bill created; exchange_parent_id = original bill id	PASS

5.6 Test Cases — Wallet, Referral, and Payment

TC ID	Test Case Description	Expected Result	Status
TC-041	Pay bill using CREDIT mode with insufficient wallet balance	400 Bad Request; insufficient credits; required vs available shown	PASS
TC-042	Pay bill using CREDIT mode with sufficient wallet balance	200 OK; wallet balance deducted; bill status = PAID; CreditTransaction(DEBIT) recorded	PASS
TC-043	View wallet balance and paginated transaction history	200 OK; current balance and full transaction list returned	PASS
TC-044	Record UPI payment transaction for a bill	PaymentTransaction created; status = PENDING; linked to bill	PASS
TC-045	Non-GST signup bonus verification	Wallet created; balance = 100; CreditTransaction (CREDIT, Signup Bonus) present	PASS
TC-046	Referral bonus: referred user makes first purchase	50 credit points added to referrer's wallet; CreditTransaction recorded	PASS

6.7 Integration Test: Complete Bill Lifecycle

This integration test scenario walks through the full lifecycle of a GST bill from seller registration to return completion, verifying that all modules interact correctly.

Step No.	Action Performed	Expected and Verified Result
1	Register new GST seller (Priti) with valid data	Account created; bill_id = priti@bill; activation email sent
2	Register new Non-GST customer (Rahul) with valid data	Account created; bill_id = rahul@bill; wallet balance = 100 credits
3	GST seller activates account via email token	is_email_verified = True; login enabled
4	GST seller completes business profile (GSTIN, address, state)	profile_completed = True; gst_verified = True
5	GST seller adds product: Laptop Bag, Rs. 1000, 18% GST, HSN 6305	Product saved with barcode; barcode lookup returns product
6	GST seller creates bill for Rahul: 2 x Laptop Bag, CASH, 5% discount	INV-0001 generated; taxable=2000; cgst=180; sgst=180; total=2242; gst_status=VALID
7	GST seller exports bill as PDF	PDF generated; contains invoice number, GSTIN, CGST, SGST rows, grand total
8	GST seller exports bill as Excel	Excel file with all item rows and summary; grand total matches
9	Seller updates bill status to PAID	Bill.payment_status = PAID; PaymentTransaction created
10	Customer raises RETURN request for 1 of 2 Laptop Bags	ReturnRequest created (PENDING); bill status = RETURN_REQUESTED
11	Seller approves return request	ReturnRequest.status = APPROVED
12	Seller completes return	CreditNote CRN-2026-001 issued; gst_reversal=180; cgst_rev=90; sgst_rev=90; bill status=PARTIALLY_RETURNED
13	Customer views credit note list	CRN-2026-001 visible with correct reversal amounts
14	Seller views GST filing summary for current month	One filing entry; total_sales=2242; cgst=180; sgst=180; bill_count=1

6.8 Integration Test: Referral and Wallet Workflow

Step No.	Action Performed	Expected and Verified Result
1	User A registers with referral code from User B	User A's referred_by = User B; wallet created with 100 credits
2	User A makes first CASH purchase (INV-0002 PAID)	Bill status = PAID; referral conversion detected
3	System awards referral bonus to User B	User B wallet balance += 50; CreditTransaction (CREDIT, Referral Bonus) created
4	User A attempts CREDIT payment; balance = 100; bill total = 150	400 Bad Request; insufficient credits; required=150, available=100
5	User A pays bill with CREDIT; balance = 200; bill total = 150	Bill PAID; wallet balance = 50; CreditTransaction(DEBIT,150) recorded
6	User A views wallet history	Two CREDIT entries (signup + referral) and one DEBIT entry visible

6.9 Performance Testing Results

API response time testing was conducted using Postman's collection runner executing 50 sequential requests per endpoint on a development machine (Intel Core i5, 8 GB RAM, SQLite database). Results represent average response times under single-user load.

API Endpoint	Avg. Response Time	Within 2s Threshold
POST /accounts/register/	318 ms	YES
POST /accounts/login/	174 ms	YES
GET /accounts/profile/	98 ms	YES
POST /billing/select-products/	210 ms	YES
POST /billing/create-bill/ (3 items)	430 ms	YES
POST /billing/create-bill/ (10 items)	780 ms	YES
GET /billing/bill/<id>/	115 ms	YES
GET /billing/export-bill-pdf/	660 ms	YES
GET /billing/export-bill-excel/	520 ms	YES
GET /billing/dashboard/	290 ms	YES
GET /billing/gst-filing/	185 ms	YES
POST /billing/complete-return/	380 ms	YES

All 12 endpoints responded within the 2-second performance requirement under single-user test conditions. PDF and Excel export endpoints were the slowest due to in-memory file generation, but both remained well within the 2-second threshold. For production deployments with PostgreSQL and Gunicorn multi-worker configuration, response times are expected to remain stable under concurrent load of up to 50 users.

5.7 Security Test Cases

TC ID	Security Test Description	Attack Simulated	Result
ST-01	Access another user's bill using valid JWT	Provide valid JWT but wrong user's bill_id	403 Forbidden — ownership check passed
ST-02	Manage product belonging to another seller	Provide another seller's product_id	404 Not Found — seller filter applied
ST-03	SQL injection attempt via bill_id field	bill_id = '1 OR 1=1'	Django ORM parameterization prevents injection
ST-04	Replay access token after expiry (60 min)	Reuse expired access token	401 Unauthorized — token rejected by SimpleJWT
ST-05	Reuse blacklisted refresh token after logout	Use refresh token post-logout	401 Unauthorized — token is blacklisted
ST-06	Modify total_amount of a PAID bill via PATCH	Send PATCH with new total_amount value	Exception raised; lock in save() blocks change
ST-07	Non-GST user attempts to create a bill	Non-GST JWT token used on create-bill endpoint	403 Forbidden — account type check failed
ST-08	Access admin panel without staff permission	Non-staff credentials on /admin/	302 Redirect to login; admin panel secured
ST-09	Register with same email twice under same account_type	Duplicate (email, account_type) combination	400 Bad Request — unique_together constraint enforced
ST-10	Attempt to activate same token twice	Re-submit already-used activation token	400 Bad Request — token deleted on first use

CHAPTER 6: USER INTERFACES

The GST Billing and Management System is built as a REST API backend. All user-facing interactions are implemented through structured JSON API endpoints. The system is designed to be consumed by any frontend client including web applications, mobile applications, or desktop software. The following section documents the key API request/response interfaces.

6.1 Registration API Interface

Endpoint: POST /api/accounts/register/

Request Body:

```
{
  'name': 'Priti Sharma',
  'email': 'priti@example.com',
  'phone': '9876543210',
  'account_type': 'GST',
  'password': 'SecurePass@123',
  'referral_code': 'REF9X4TK2' // optional
}
```

Success Response (201 Created):

```
{
  'success': true,
  'message': 'User registered. Check email for activation link.',
  'data': {
    'bill_id': 'priti@bill',
    'referral_code': 'REF7B2MX9',
    'referral_link': 'http://domain.com/register/?ref=REF7B2MX9',
    'email_sent': true
  }
}
```

6.2 Login API Interface

Endpoint: POST /api/accounts/login/

Request Body:

```
{ 'email': 'priti@example.com', 'password': 'SecurePass@123',
  'account_type': 'GST' }
```

Success Response (200 OK):

```
{
  'access': 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... ',
  'refresh': 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... ',
  'user': {
    'id': 'uuid-abc123',
    'name': 'Priti Sharma',
    'email': 'priti@example.com',
    'account_type': 'GST',
    'bill_id': 'priti@bill',
    'gst_verified': true,
    'profile_completed': true
  }
}
```

6.3 Create Bill API Interface

Endpoint: POST /api/billing/create-bill/ (requires JWT Authorization header)

```
Request Body:
{
  'bill_id': 'customer@bill',
  'items': [
    { 'product_id': 5, 'quantity': 2, 'price': 1000.00, 'gst_percentage':
18 }
  ],
  'payment_mode': 'CASH',
  'discount': 5
}

Success Response (201 Created):
{
  'invoice_number'      : 'INV-0042',
  'total_amount'        : 2242.00,
  'total_taxable_value' : 2000.00,
  'total_gst'           : 360.00,
  'total_cgst'          : 180.00,
  'total_sgst'          : 180.00,
  'discount_percentage' : 5.0,
  'discount_amount'     : 118.00,
  'gst_status'          : 'VALID',
  'is_flagged'          : false
}
```

6.4 Bill Detail API Interface

Endpoint: GET /api/billing/bill/<bill_id>/

```
Response (200 OK):
{
  'invoice_number' : 'INV-0042',
  'seller' : { 'name': 'Priti Enterprises', 'gstin': '07AAPZ1234A1Z5' },
  'customer' : { 'name': 'Rahul Kumar', 'bill_id': 'rahul@bill' },
  'items': [
    {
      'name'      : 'Laptop Bag',
      'quantity'  : 2,
      'price'     : 1000.0,
      'taxable_value' : 2000.0,
      'cgst'      : 180.0,
      'sgst'      : 180.0,
      'total_price' : 2360.0
    }
  ],
  'total_cgst'      : 180.0,
  'total_sgst'      : 180.0,
  'total_amount'    : 2242.0,
  'payment_status'  : 'PAID',
  'gst_status'      : 'VALID'
}
```

6.5 GST Filing Summary API Interface

Endpoint: GET /api/billing/gst-filing/ (GST sellers only)

Response (200 OK):

```
[
  { 'month_year': '2026-04', 'total_sales': 125400.0,
    'cgst': 11286.0, 'sgst': 11286.0, 'bill_count': 42 },
  { 'month_year': '2026-03', 'total_sales': 98750.0,
    'cgst': 8887.5, 'sgst': 8887.5, 'bill_count': 35 },
  { 'month_year': '2026-02', 'total_sales': 112300.0,
    'cgst': 10107.0, 'sgst': 10107.0, 'bill_count': 48 }
]
```

6.6 Credit Note API Interface

Endpoint: GET /api/billing/my-credit-notes/

Response (200 OK):

```
[
  {
    'credit_note_number' : 'CRN-2026-001',
    'original_invoice'   : 'INV-0042',
    'taxable_reversal'   : 1000.0,
    'gst_reversal'       : 180.0,
    'cgst_reversal'      : 90.0,
    'sgst_reversal'      : 90.0,
    'created_at'         : '2026-04-29T10:23:45Z'
  }
]
```

6.7 Return Request API Interface

Endpoint: POST /api/billing/create-return-request/ (JWT required)

Request Body:

```
{
  'original_bill_id' : 'uuid-of-paid-bill',
  'request_type'     : 'RETURN',
  'reason'           : 'Product received was damaged during delivery',
  'items': [
    { 'bill_item_id': 7, 'quantity': 1 }
  ]
}
```

Success Response (201 Created):

```
{
  'return_request_id' : 12,
  'status'            : 'PENDING',
  'bill_status_updated_to': 'RETURN_REQUESTED',
  'message': 'Return request submitted. Seller has been notified.'
}
```

Error Response (400 Bad Request – bill not PAID):

```
{
  'error': 'Return requests can only be raised on PAID bills.'
}
```

6.8 Wallet API Interface

Endpoint: GET /api/accounts/wallet/ (JWT required)

Response (200 OK):

```
{
  'balance': 175,
  'transactions': [
    { 'id':1, 'type':'CREDIT', 'amount':100,
      'reason':'Signup Bonus', 'date':'2026-01-15' },
    { 'id':2, 'type':'CREDIT', 'amount':50,
      'reason':'Referral Bonus', 'date':'2026-02-10' },
    { 'id':3, 'type':'DEBIT', 'amount':250,
      'reason':'Payment INV-0021', 'date':'2026-03-05' },
    { 'id':4, 'type':'CREDIT', 'amount':275,
      'reason':'Refund CRN-2026-003', 'date':'2026-03-08' }
  ]
}
```

6.9 Dashboard API Interface

Endpoint: GET /api/billing/dashboard/ (GST sellers only)

Response (200 OK):

```
{
  'total_sales'      : 336150.00,
  'total_cgst_collected' : 30253.50,
  'total_sgst_collected' : 30253.50,
  'total_bills'      : 125,
  'pending_bills'    : 8,
  'flagged_bills'    : 2,
  'this_month': {
    'sales'      : 125400.00,
    'cgst'       : 11286.00,
    'sgst'       : 11286.00,
    'bill_count' : 42
  },
  'recent_bills': [
    { 'invoice_number':'INV-0125', 'customer':'Rahul Kumar',
      'amount':2242.00, 'status':'PAID', 'date':'29-04-2026' },
    { 'invoice_number':'INV-0124', 'customer':'Sneha Patel',
      'amount':1575.00, 'status':'PAID', 'date':'28-04-2026' }
  ]
}
```

6.10 Notifications API Interface

Endpoint: GET /api/accounts/notifications/ (JWT required)

Response (200 OK):

```
{
  'unread_count': 2,
  'notifications': [
    {
      'id'      : 45,
      'title'   : 'Bill Verification Request',
      'message' : 'Seller Priti requests approval for INV-0042
                  of Rs. 2242.00',
      'is_read' : false,
      'created_at': '2026-04-29T14:30:22Z'
    },
    {
      'id'      : 44,
      'title'   : 'Return Request Approved',
      'message' : 'Your return for INV-0035 is approved.
                  Credit note CRN-2026-007 has been issued.',
      'is_read' : false,
      'created_at': '2026-04-28T10:15:00Z'
    }
  ]
}
```

CHAPTER 7: APPENDICES

Appendix A: Complete API Endpoint Reference

Endpoint	Method / Auth Required	Description
/api/accounts/register/	POST / No	Register new user (GST or Non-GST)
/api/accounts/activate/<token>/	GET / No	Activate account via email token
/api/accounts/login/	POST / No	Login and receive JWT access + refresh tokens
/api/accounts/logout/	POST / Yes	Logout and blacklist the refresh token
/api/accounts/forgot-password/	POST / No	Send password reset email to user
/api/accounts/reset-password/<token>/	POST / No	Reset password using email token
/api/accounts/profile/	GET, PUT / Yes	View or update basic user profile
/api/accounts/gst-profile/	GET, PUT / Yes	View or update GST business profile
/api/accounts/wallet/	GET / Yes	View current wallet credit balance
/api/accounts/wallet-history/	GET / Yes	Paginated credit transaction history
/api/accounts/my-referral/	GET / Yes	View referral code and referral count
/api/accounts/notifications/	GET / Yes	View user notification list
/api/billing/select-products/	POST / Yes	Add or update a product in catalog
/api/billing/my-products/	GET / Yes	List all products for the seller
/api/billing/product-by-barcode/	GET / Yes	Get product details by barcode scan
/api/billing/create-bill/	POST / Yes	Create a new GST-compliant invoice
/api/billing/bill/<bill_id>/	GET / Yes	Get full bill detail with items
/api/billing/bill-status/	POST / Yes	Update bill payment status
/api/billing/my-sales/	GET / Yes	Seller's complete sales history
/api/billing/my-purchases/	GET / Yes	Customer's complete purchase history
/api/billing/export-bill-pdf/	GET / Yes	Download invoice as GST-compliant PDF
/api/billing/export-bill-excel/	GET / Yes	Download invoice as Excel spreadsheet
/api/billing/send-verification-notification/	POST / Yes	Request buyer bill verification
/api/billing/verify-bill-request/	POST / Yes	Buyer approves or rejects verification

/api/billing/create-return-request/	POST / Yes	Raise return/exchange request
/api/billing/approve-return/	POST / Yes	Seller approves a return request
/api/billing/reject-return/	POST / Yes	Seller rejects a return request
/api/billing/complete-return/	POST / Yes	Complete return and issue credit note
/api/billing/my-credit-notes/	GET / Yes	List all credit notes issued
/api/billing/dashboard/	GET / Yes	Seller analytics dashboard summary
/api/billing/gst-filing/	GET / Yes	Monthly GST filing summary by period
/api/payments/initiate/	POST / Yes	Initiate a payment transaction

Appendix B: GST Computation Examples

Example 1: Electronic Item (HSN: 8517, GST Rate: 18%)

Field	Value
Product Price per Unit	Rs. 15,000
Quantity	2
Taxable Value (Qty x Price)	Rs. 30,000
GST Amount @ 18%	Rs. 5,400
CGST @ 9% (GST/2)	Rs. 2,700
SGST @ 9% (GST/2)	Rs. 2,700
Grand Total (Taxable + GST)	Rs. 35,400

Example 2: Garment Item (HSN: 6203, GST Rate: 5%)

Field	Value
Product Price per Unit	Rs. 800
Quantity	3
Taxable Value	Rs. 2,400
GST Amount @ 5%	Rs. 120
CGST @ 2.5%	Rs. 60
SGST @ 2.5%	Rs. 60
Grand Total	Rs. 2,520

Example 3: Bill with 5% Discount Applied

Field	Value
Total Before Discount	Rs. 35,400
Discount Percentage	5%
Discount Amount (Total x 5%)	Rs. 1,770
Final Grand Total	Rs. 33,630

Appendix C: Project File Structure

```

gst_project/
├── accounts/
│   ├── models.py          # CustomUser, Wallet,
│   │                       CreditTransaction, Notification
│   └── views.py           # Register, Login, Profile, Wallet,
│   │                       Referral APIs
│   └── serializers.py     # Request validation and response
│   │                       serialization
│   └── emails.py          # Activation and reset email
│   └── dispatch functions
│       ├── utils.py       # GSTIN format validation helpers
│       └── urls.py        # accounts/ URL routing
├── billing/
│   ├── models.py         # Bill, Product, BillItem, Return,
│   │                       CreditNote, GSTFiling
│   └── views.py          # Billing, Return, Export,
│   │                       Dashboard, Filing APIs
│   ├── utils.py          # validate_gst() function
│   ├── admin.py          # Django admin panel registrations
│   └── urls.py           # billing/ URL routing
├── payments/
│   ├── models.py         # PaymentTransaction
│   └── views.py          # Payment initiation and
│   │                       confirmation APIs
│   └── urls.py           # payments/ URL routing
├── gst_project/
│   ├── settings.py       # Django project-wide configuration
│   └── urls.py           # Root URL dispatcher (includes all
│   │                       app URLs)
│   ├── asgi.py           # ASGI server entry point
│   └── wsgi.py           # WSGI server entry point
├── media/
│   └── qr_codes/         # Generated QR code images for Non-
│   │                       GST users
│   └── barcodes/         # Generated barcode images for Non-
│   │                       GST users
│   └── products/         # Product images uploaded by sellers
└── manage.py             # Django management command-line
interface

```

Appendix D: Bill Lifecycle Status Transitions

Status	Description
--------	-------------

CREATED	Bill has been created but not yet paid
PAID	Payment has been confirmed and recorded
RETURN_REQUESTED	Customer has raised a return request
RETURNED	All items returned; credit note issued
PARTIALLY_RETURNED	Some items returned; partial credit note issued
EXCHANGED	Items exchanged; new exchange bill created
REVERSED	Unpaid bill reversed by seller before payment

Appendix E: Sample GST Invoice Layout

Invoice Field	Sample Value
Document Title	TAX INVOICE
Invoice Number	INV-0042
Invoice Date	29-04-2026
Supplier (Seller) Name	Priti Enterprises
Supplier GSTIN	07AAPZ1234A1Z5
Supplier Address	Shop No. 12, Connaught Place, New Delhi — 110001
Supplier State	Delhi
Recipient (Buyer) Name	Rahul Kumar
Recipient Bill ID	rahul@bill
Place of Supply	Delhi (Intra-state)
HSN Code	6305 (Sacks and Bags)
Item Description	Laptop Bag — Brand X
Quantity	2 NOS
Unit Price (Ex-GST)	Rs. 1,000.00
Taxable Value	Rs. 2,000.00
CGST Rate	9%
CGST Amount	Rs. 180.00
SGST Rate	9%
SGST Amount	Rs. 180.00
Total GST	Rs. 360.00
Discount (5%)	Rs. 118.00
Grand Total	Rs. 2,242.00
Amount in Words	Two Thousand Two Hundred Forty-Two Rupees Only
Payment Mode	CASH

GST Validation Status	VALID
-----------------------	-------

Appendix F: Installation and Setup Guide

```
# Step 1: Clone the project repository
git clone https://github.com/username/gst-billing-system.git
cd gst-billing-system

# Step 2: Create and activate a Python virtual environment
python -m venv env
source env/bin/activate          # Linux / macOS
env\Scripts\activate            # Windows

# Step 3: Install all dependencies
pip install -r requirements.txt

# Step 4: Create .env file with required environment variables
SECRET_KEY=your-django-secret-key-here
DEBUG=True
DATABASE_URL=sqlite:///db.sqlite3
EMAIL_HOST=smtp.gmail.com
EMAIL_HOST_USER=your@gmail.com
EMAIL_HOST_PASSWORD=your-app-password
FRONTEND_URL=http://localhost:3000

# Step 5: Apply database migrations
python manage.py makemigrations accounts billing payments
python manage.py migrate

# Step 6: Create a superuser for the admin panel
python manage.py createsuperuser

# Step 7: Start the development server
python manage.py runserver

# API base URL: http://127.0.0.1:8000/api/
# Admin panel: http://127.0.0.1:8000/admin/
```

Appendix G: requirements.txt

```
Django==4.2.7
django-rest-framework==3.14.0
django-rest-framework-simplejwt==5.3.0
django-cors-headers==4.3.0
dj-database-url==2.1.0
reportlab==4.0.7
openpyxl==3.1.2
pandas==2.1.3
Pillow==10.1.0
qrcode==7.4.2
python-barcode==0.15.1
python-decouple==3.8
psycpg2-binary==2.9.9
gunicorn==21.2.0
```

Appendix H: Glossary of Terms

Term	Definition
GST	Goods and Services Tax — unified indirect tax in India since 1st July 2017
CGST	Central Goods and Services Tax — central government's share of intra-state GST
SGST	State Goods and Services Tax — state government's share of intra-state GST
IGST	Integrated GST — levied on inter-state supplies; not in scope of this project
HSN Code	Harmonised System of Nomenclature — 4 to 8 digit code classifying goods for GST
SAC Code	Services Accounting Code — classifies services for GST purposes
GSTIN	GST Identification Number — unique 15-character alphanumeric ID for GST businesses
Credit Note	Document issued by seller acknowledging return of goods with GST reversal details
Bill ID	Unique digital billing address for each user e.g. priti@bill
JWT	JSON Web Token — compact token format for stateless API authentication
REST API	Representational State Transfer — architecture style for HTTP-based web APIs
CGST Reversal	Amount of CGST refunded/reversed when goods are returned by buyer
SGST Reversal	Amount of SGST refunded/reversed when goods are returned by buyer
Invoice Number	Sequential bill identifier in format INV-XXXX auto-assigned by the system
Taxable Value	Price of goods before GST; basis for GST computation = Qty x Unit Price
UPI	Unified Payments Interface — India's real-time digital payment system
E-invoice	Electronically generated invoice compliant with GSTN standards
GSTN	Goods and Services Tax Network — IT backbone of India's GST infrastructure
Django ORM	Object-Relational Mapper — Django's database abstraction layer using Python classes
Wallet	Digital credit points account for Non-GST users; used for payments and referral rewards
Referral Code	Unique alphanumeric code assigned to each user for the referral rewards program
Bill Lifecycle	Sequence of states a bill passes through: CREATED → PAID → RETURNED/EXCHANGED

Appendix I: Future Enhancements

Enhancement	Description
IGST Support	Add inter-state transaction support with IGST computation for cross-state billing
GSTN E-Invoice Integration	Direct API integration with GSTN portal for e-invoice generation and IRN (Invoice Reference Number) fetching
Mobile Application	Native Android and iOS apps consuming the existing REST API for on-the-go billing
Barcode Scanner Integration	Camera-based barcode scanning in the mobile app to auto-fill product details
Automated GST Filing	Auto-populate GSTR-1 and GSTR-3B forms from monthly filing summaries for one-click submission
Multi-branch Support	Allow a single GST seller to manage multiple business branches under one account
Inventory Management	Track product stock levels, auto-alert on low stock, and prevent over-billing
Recurring Bills	Support subscription-based billing with auto-generation of periodic invoices
Customer Credit Limit	Allow sellers to set per-customer credit limits for CREDIT payment mode
Advanced Analytics	Charts and graphs for sales trends, top products, and GST liability forecasting
WhatsApp Bill Sharing	Send invoice PDF directly to customer's WhatsApp via WhatsApp Business API
Multi-currency Support	Enable billing in currencies other than INR for exporters

Appendix J: Project Development Timeline

Phase	Duration	Activities Completed
Phase 1: Planning and Analysis	Week 1-2	Requirement gathering, feasibility study, technology selection, project scope definition
Phase 2: System Design	Week 3-4	Database schema design, ER diagram, DFD, flowcharts, API endpoint planning, architecture design
Phase 3: accounts App Development	Week 5-6	CustomUser model, JWT auth, email verification, wallet, referral, QR/barcode generation
Phase 4: billing App Development	Week 7-9	Product model, Bill model, GST computation, PDF export, Excel export, return management, credit notes
Phase 5: payments App Development	Week 10	PaymentTransaction model, multi-mode payment support, CREDIT deduction logic

Phase 6: Integration and Testing	Week 11-12	Cross-module integration, unit testing, integration testing, security testing, performance testing via Postman
Phase 7: Documentation	Week 13-14	Project report writing, API documentation, deployment guide, code comments and docstrings

CHAPTER 8: REFERENCES AND BIBLIOGRAPHY

1. Django Software Foundation (2024). *Django Documentation, Version 4.2*.
<https://docs.djangoproject.com/en/4.2/>

2. Encode OSS Ltd. (2024). *Django REST Framework Documentation*.
<https://www.django-rest-framework.org/>
3. SimpleJWT (2024). *Django REST Framework SimpleJWT Documentation*.
<https://django-rest-framework-simplejwt.readthedocs.io/>
4. ReportLab Inc. (2024). *ReportLab PDF Generation Library*.
<https://www.reportlab.com/docs/>
5. Government of India, Ministry of Finance (2023). *Goods and Services Tax Act, 2017 and Rules*. <https://www.gst.gov.in/>
6. GSTN (Goods and Services Tax Network) (2023). *GST HSN Code Master Database*. <https://www.gst.gov.in/>
7. Openpyxl (2024). *A Python library to read and write Excel 2010 xlsx/xlsm files*. <https://openpyxl.readthedocs.io/>
8. Pandas Development Team (2024). *Pandas: Python Data Analysis Library*. <https://pandas.pydata.org/docs/>
9. Python Software Foundation (2024). *Python 3.9 Documentation*.
<https://docs.python.org/3.9/>
10. python-barcode (2024). *Pure Python barcode generation library*.
<https://python-barcode.readthedocs.io/>
11. qrcode (2024). *Pure Python QR Code Generator*.
<https://pypi.org/project/qrcode/>
12. Pressman, R. S. and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach, 9th Edition*. McGraw-Hill Education.
13. Sommerville, I. (2016). *Software Engineering, 10th Edition*. Pearson Education.
14. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.